

Stage 1 – Create the Employees Table

Database and Table Creation

explanation: created the collage_db database and used it. created the employees table with employee details and applied constraints such as primary key,auto_increment,not null,unique,check,and default.

The screenshot shows a SQL IDE window titled "SQL File 4* x". The toolbar includes icons for file operations, execution, and navigation. The SQL editor contains the following code:

```
1 • create database collage_db;
2 • use collage_db;
3 • create table employees(
4     emp_id int auto_increment primary key,
5     emp_name varchar(50) not null,
6     email varchar(50) not null unique,
7     department varchar(50) not null,
8     job_role varchar(30) not null,
9     salary decimal(10,2) not null check(salary >0),
10    age int not null check(age>=18),
11    city varchar(50) default "hyderabad");
```

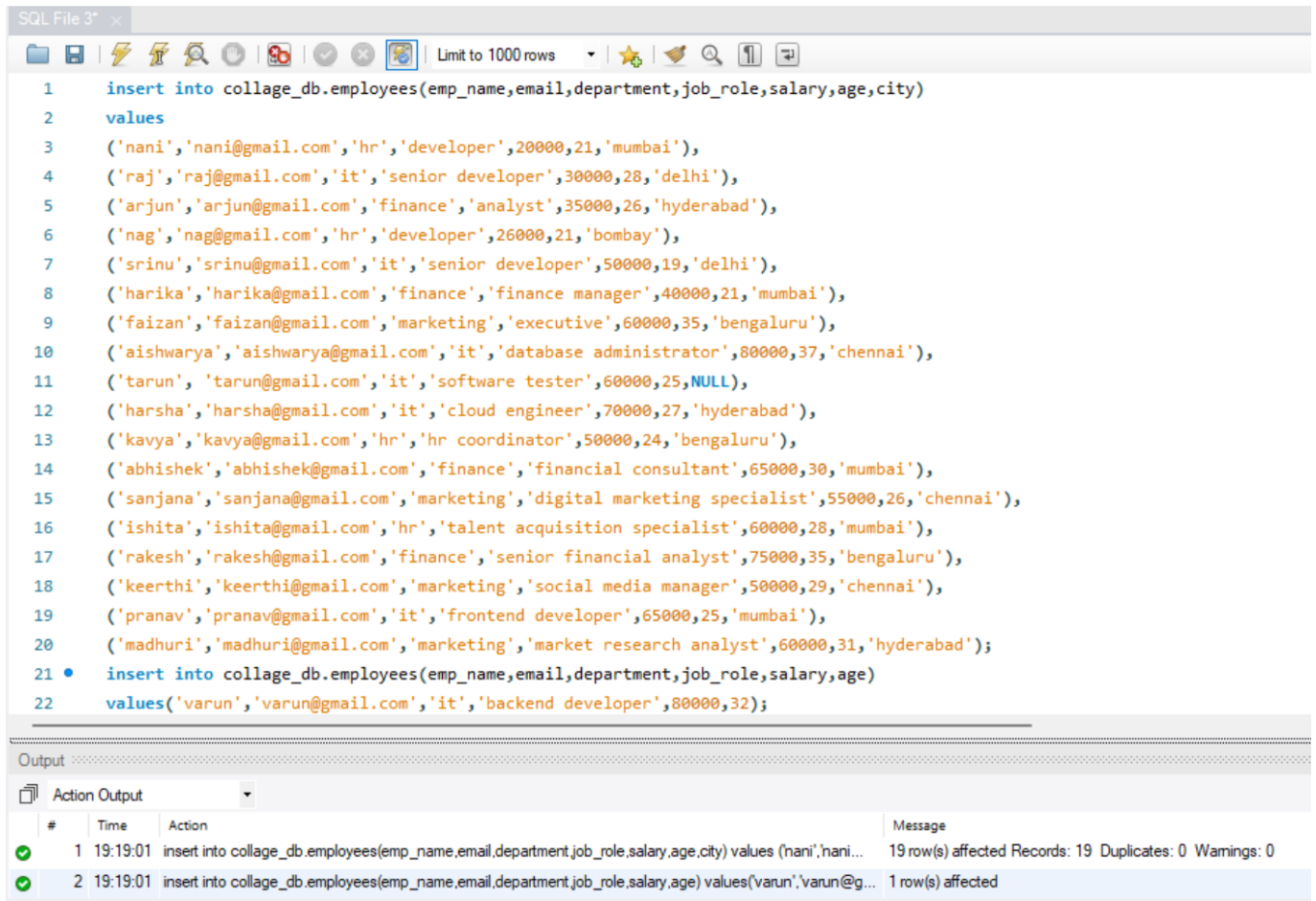
Below the editor is the "Output" panel, which shows the execution results in a table:

#	Time	Action	Message
✓ 1	13:41:26	create database collage_db	1 row(s) affected
✓ 2	13:41:26	use collage_db	0 row(s) affected
✓ 3	13:41:26	create table employees(emp_id int auto_increment primary key, emp_name varchar(50) not n...	0 row(s) affected

Stage 2 – Insert Employee Data

Inserting Employee Records

explanation: inserted 20 employee records into the employees table, including different departments, cities, job roles, salaries and ages. the data also includes repeated salaries, repeated cities, and a null city value. in the 20th record, the city was not specified, so the default value hyderabad was used.



```
1  insert into collage_db.employees(emp_name,email,department,job_role,salary,age,city)
2  values
3  ('nani','nani@gmail.com','hr','developer',20000,21,'mumbai'),
4  ('raj','raj@gmail.com','it','senior developer',30000,28,'delhi'),
5  ('arjun','arjun@gmail.com','finance','analyst',35000,26,'hyderabad'),
6  ('nag','nag@gmail.com','hr','developer',26000,21,'bombay'),
7  ('srinu','srinu@gmail.com','it','senior developer',50000,19,'delhi'),
8  ('harika','harika@gmail.com','finance','finance manager',40000,21,'mumbai'),
9  ('faizan','faizan@gmail.com','marketing','executive',60000,35,'bengaluru'),
10 ('aishwarya','aishwarya@gmail.com','it','database administrator',80000,37,'chennai'),
11 ('tarun','tarun@gmail.com','it','software tester',60000,25,NULL),
12 ('harsha','harsha@gmail.com','it','cloud engineer',70000,27,'hyderabad'),
13 ('kavya','kavya@gmail.com','hr','hr coordinator',50000,24,'bengaluru'),
14 ('abhishek','abhishek@gmail.com','finance','financial consultant',65000,30,'mumbai'),
15 ('sanjana','sanjana@gmail.com','marketing','digital marketing specialist',55000,26,'chennai'),
16 ('ishita','ishita@gmail.com','hr','talent acquisition specialist',60000,28,'mumbai'),
17 ('rakesh','rakesh@gmail.com','finance','senior financial analyst',75000,35,'bengaluru'),
18 ('keerthi','keerthi@gmail.com','marketing','social media manager',50000,29,'chennai'),
19 ('pranav','pranav@gmail.com','it','frontend developer',65000,25,'mumbai'),
20 ('madhuri','madhuri@gmail.com','marketing','market research analyst',60000,31,'hyderabad');
21 • insert into collage_db.employees(emp_name,email,department,job_role,salary,age)
22 values('varun','varun@gmail.com','it','backend developer',80000,32);
```

Output

Action Output

#	Time	Action	Message
✓ 1	19:19:01	insert into collage_db.employees(emp_name,email,department,job_role,salary,age,city) values ('nani','nani...	19 row(s) affected Records: 19 Duplicates: 0 Warnings: 0
✓ 2	19:19:01	insert into collage_db.employees(emp_name,email,department,job_role,salary,age) values('varun','varun@g...	1 row(s) affected

Stage 3 – SELECT Queries

Displaying All Employee Records

explanation: used the select * statement to display all records from the employees table and verify the inserted employee data.

SQL File 3* x

Limit to 1000 rows

```
1 • select * from collage_db.employees;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

	emp_id	emp_name	email	department	job_role	salary	age	city
▶	1	nani	nani@gmail.com	hr	developer	20000.00	21	mumbai
	2	raj	raj@gmail.com	it	senior developer	30000.00	28	delhi
	3	arjun	arjun@gmail.com	finance	analyst	35000.00	26	hyderabad
	4	nag	nag@gmail.com	hr	developer	26000.00	21	bombay
	5	srinu	srinu@gmail.com	it	senior developer	50000.00	19	delhi
	6	harika	harika@gmail.com	finance	finance manager	40000.00	21	mumbai
	7	faizan	faizan@gmail.com	marketing	executive	60000.00	35	bengaluru
	8	nikhil	nikhil@gmail.com	hr	training	50000.00	29	bengaluru
	9	aishwarya	aishwarya@gmail.com	it	database administrator	80000.00	37	chennai
	10	tarun	tarun@gmail.com	it	software tester	60000.00	25	NULL
	11	harsha	harsha@gmail.com	it	cloud engineer	70000.00	27	hyderabad
	12	kavya	kavya@gmail.com	hr	hr coordinator	50000.00	24	bengaluru
	13	abhishek	abhishek@gmail.com	finance	financial consultant	65000.00	30	mumbai
	14	sanjana	sanjana@gmail.com	marketing	digital marketing speci...	55000.00	26	chennai
	15	ishita	ishita@gmail.com	hr	talent acquisition speci...	60000.00	28	mumbai
	16	rakesh	rakesh@gmail.com	finance	senior financial analyst	75000.00	35	bengaluru
	17	keerthi	keerthi@gmail.com	marketing	social media manager	50000.00	29	chennai
	18	pranav	pranav@gmail.com	it	frontend developer	65000.00	25	mumbai
	19	madhuri	madhuri@gmail.com	marketing	market research analyst	60000.00	31	hyderabad
	20	varun	varun@gmail.com	it	backend developer	80000.00	32	hyderabad

Stage 3 – SELECT Queries

Selecting Specific Columns

explanation: used select to display specific employee details such as employee name,department,salary,and city.

```
SQL File 3* x
Limit to 1000 rows
1 • select emp_name,department,salary,city from collage_db.employees;
```

Result Grid Filter Rows: Export: Wrap Cell Content:				
	emp_name	department	salary	city
▶	nani	hr	20000.00	mumbai
	raj	it	30000.00	delhi
	arjun	finance	35000.00	hyderabad
	nag	hr	26000.00	bombay
	srinu	it	50000.00	delhi
	harika	finance	40000.00	mumbai
	faizan	marketing	60000.00	bengaluru
	nikhil	hr	50000.00	bengaluru
	aishwarya	it	80000.00	chennai
	tarun	it	60000.00	NULL
	harsha	it	70000.00	hyderabad
	kavya	hr	50000.00	bengaluru
	abhishek	finance	65000.00	mumbai
	sanjana	marketing	55000.00	chennai
	ishita	hr	60000.00	mumbai
	rakesh	finance	75000.00	bengaluru
	keerthi	marketing	50000.00	chennai
	pranav	it	65000.00	mumbai
	madhuri	marketing	60000.00	hyderabad
	varun	it	80000.00	hyderabad

employees 1 x

Stage 4 – WHERE and Comparison Operators

WHERE with Equal To (=)

explanation: used the where clause to display employees whose salary is equal to rs.60,000.

```
SQL File 3* x
Limit to 1000 rows
1 select emp_name,salary from collage_db.employees where salary=60000;
2
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	emp_name	salary			
▶	faizan	60000.00			
	tarun	60000.00			
	ishita	60000.00			
	madhuri	60000.00			

Stage 4 – WHERE and Comparison Operators

WHERE with Greater Than (>)

explanation: used the where clause to display employees whose salary is greater than rs.50,000.

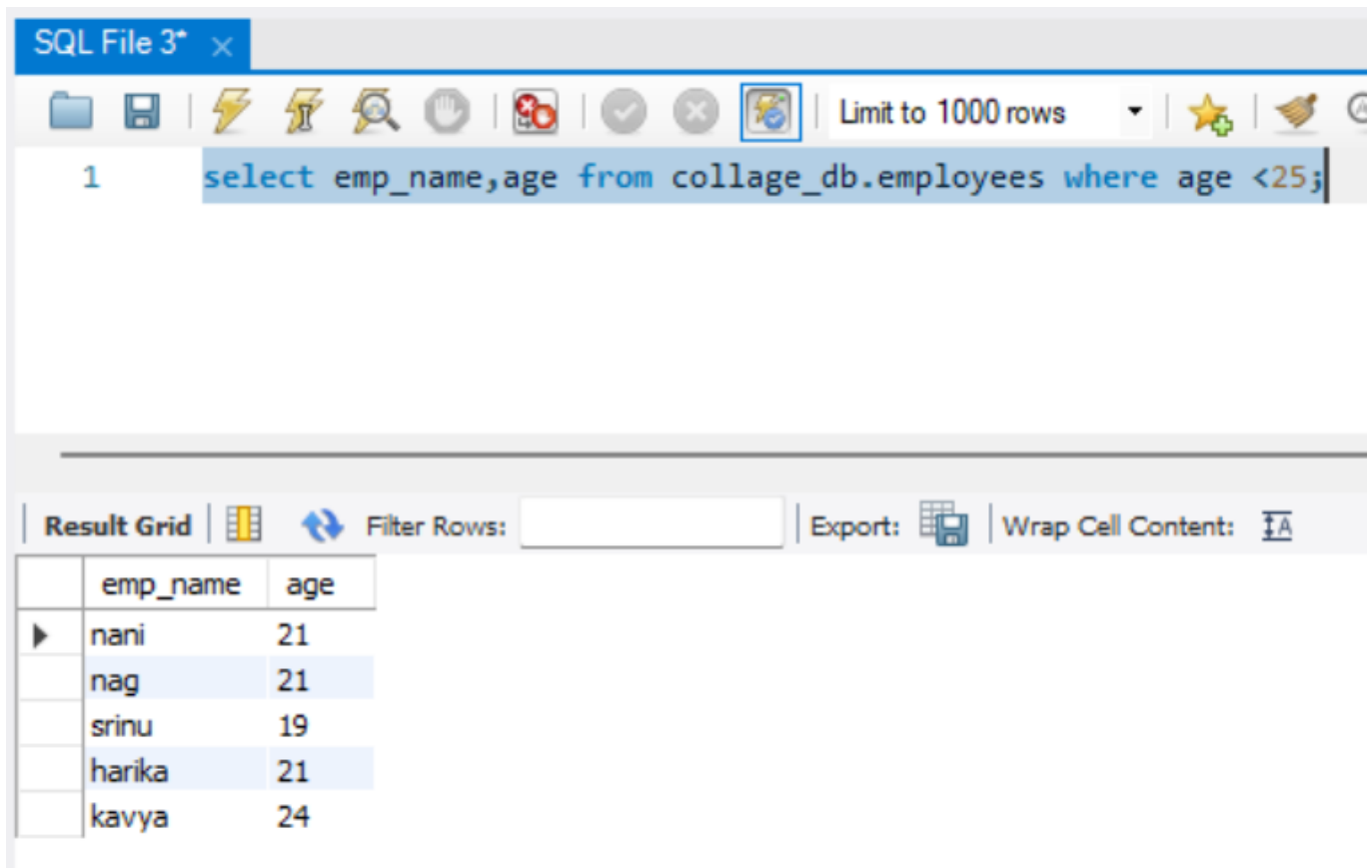
```
SQL File 3* x
Limit to 1000 rows
1 • select emp_name,salary from collage_db.employees where salary > 50000;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	emp_name	salary			
▶	faizan	60000.00			
	aishwarya	80000.00			
	tarun	60000.00			
	harsha	70000.00			
	abhishek	65000.00			
	sanjana	55000.00			
	ishita	60000.00			
	rakesh	75000.00			
	pranav	65000.00			
	madhuri	60000.00			
	varun	80000.00			

Stage 4 – WHERE and Comparison Operators

WHERE with Less Than (<)

explanation: used the where clause to display employees whose age is less than 25 years.



The screenshot shows a SQL IDE window titled "SQL File 3* x". The query editor contains the following SQL statement:

```
1 select emp_name,age from collage_db.employees where age <25;
```

The results are displayed in a table with the following data:

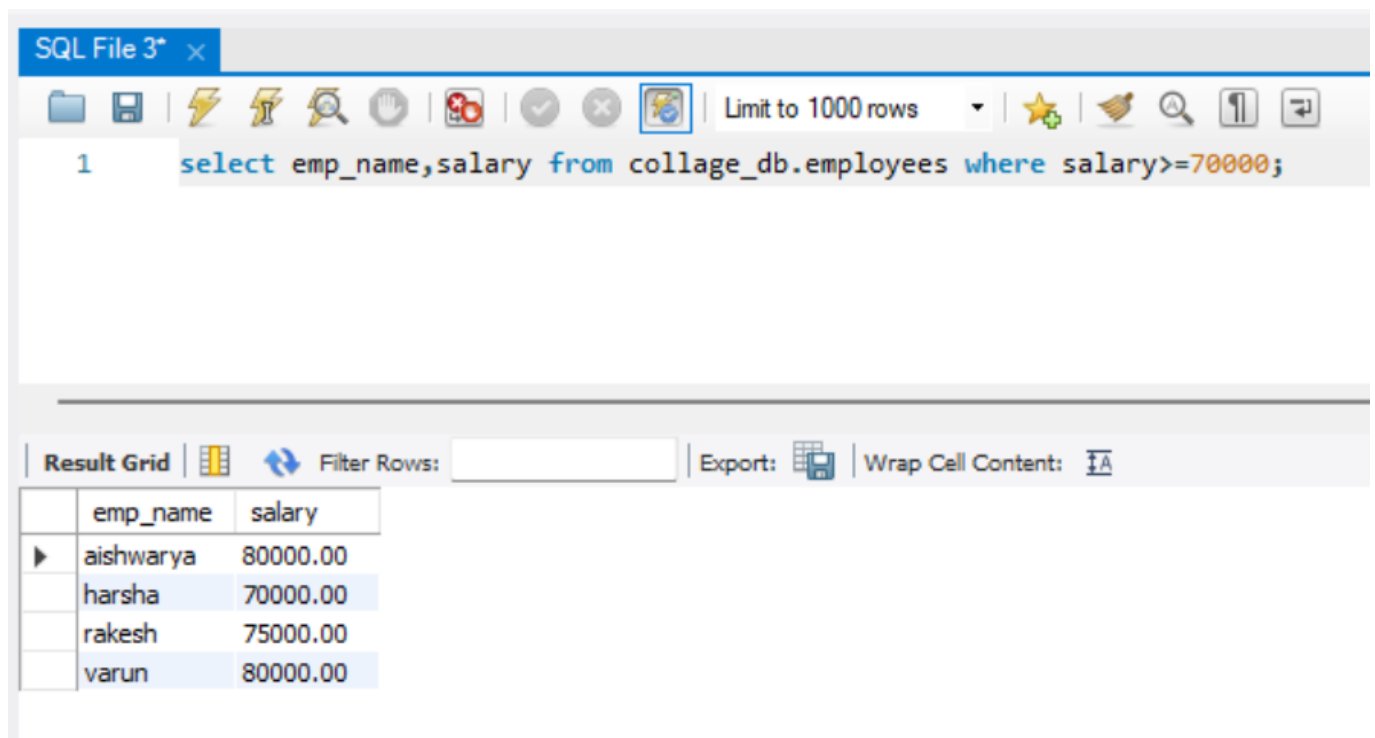
	emp_name	age
▶	nani	21
	nag	21
	srinu	19
	harika	21
	kavya	24

The IDE interface includes a toolbar with various icons for file operations, execution, and settings. The "Result Grid" tab is active, and the "Filter Rows" field is empty. The "Export" button is visible, and the "Wrap Cell Content" option is checked.

Stage 4 – WHERE and Comparison Operators

WHERE with Greater Than or Equal To (>=)

explanation: used the where clause to display employees whose salary is greater than or equal to rs.70,000.



The screenshot shows a SQL IDE interface. At the top, a tab labeled "SQL File 3*" is open. Below the tab is a toolbar with various icons. The main text area contains the following SQL query:

```
1 select emp_name,salary from collage_db.employees where salary>=70000;
```

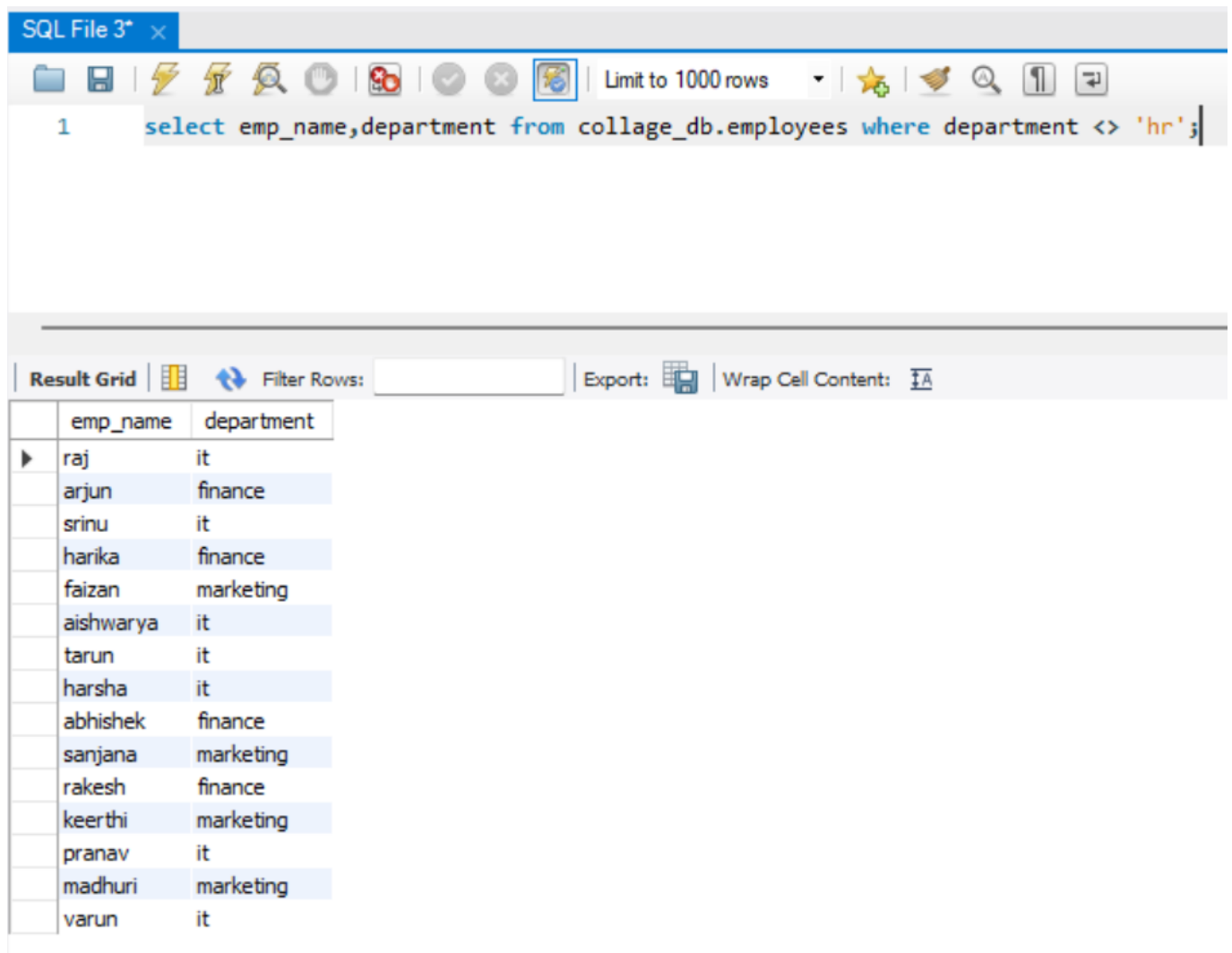
Below the query editor, there is a "Result Grid" section. It includes a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. The result grid displays the following data:

	emp_name	salary
▶	aishwarya	80000.00
	harsha	70000.00
	rakesh	75000.00
	varun	80000.00

Stage 4 – WHERE and Comparison Operators

WHERE with Not Equal To (<>)

explanation: used the <> operator to display employees who are not from the hr department.



The screenshot shows a SQL IDE interface. At the top, a tab labeled 'SQL File 3*' is open. Below the tab is a toolbar with various icons for file operations, editing, and execution. A dropdown menu shows 'Limit to 1000 rows'. The main editor area contains the following SQL query:

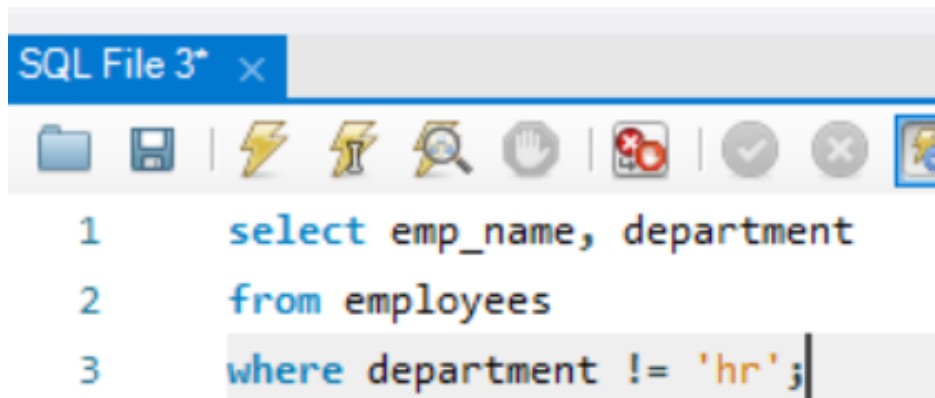
```
1 select emp_name,department from collage_db.employees where department <> 'hr';
```

Below the editor, the 'Result Grid' is displayed, showing the output of the query. The grid has two columns: 'emp_name' and 'department'. The results are as follows:

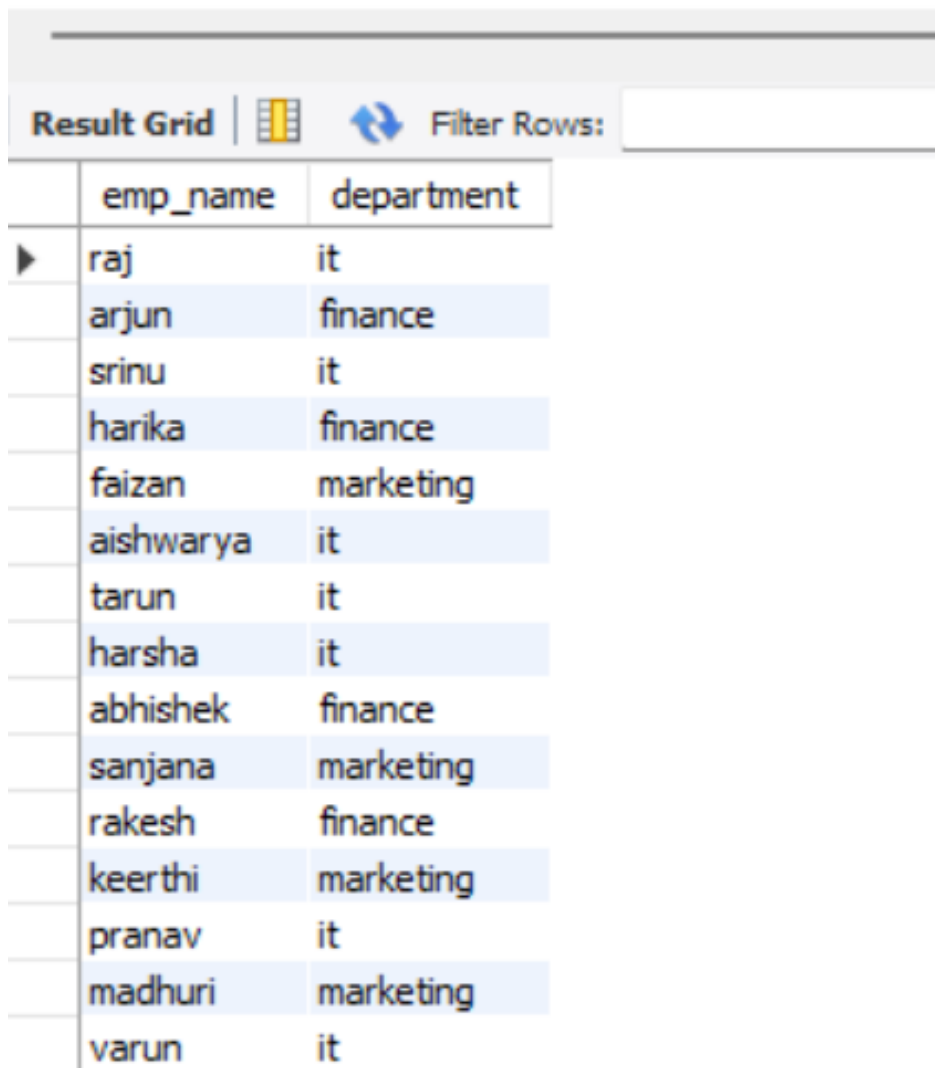
emp_name	department
raj	it
arjun	finance
srinu	it
harika	finance
faizan	marketing
aishwarya	it
tarun	it
harsha	it
abhishek	finance
sanjana	marketing
rakesh	finance
keerthi	marketing
pranav	it
madhuri	marketing
varun	it

WHERE with Not Equal To (!=)

explanation: used the != operator to display employees who are not from the hr department.



```
1  select emp_name, department
2  from employees
3  where department != 'hr';
```

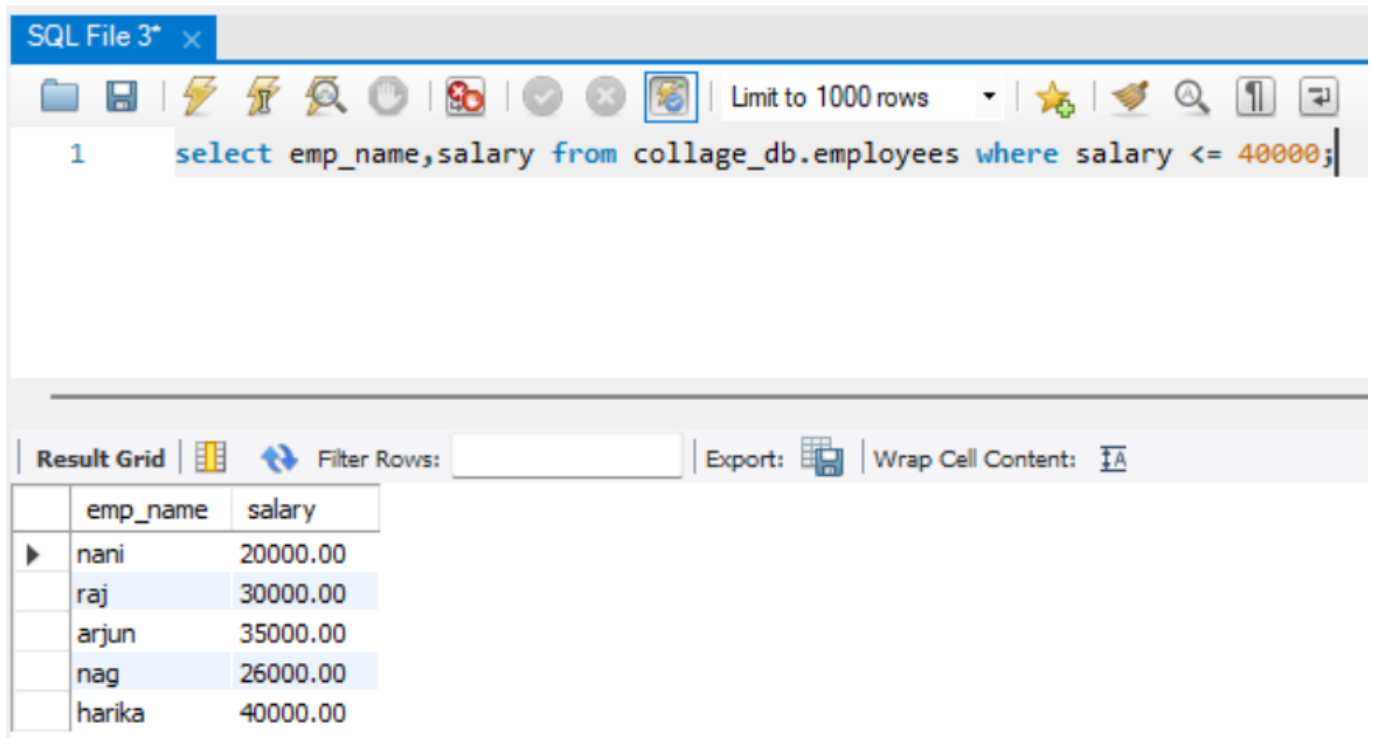


	emp_name	department
▶	raj	it
	arjun	finance
	srinu	it
	harika	finance
	faizan	marketing
	aishwarya	it
	tarun	it
	harsha	it
	abhishek	finance
	sanjana	marketing
	rakesh	finance
	keerthi	marketing
	pranav	it
	madhuri	marketing
	varun	it

Stage 4 – WHERE and Comparison Operators

WHERE with Less Than or Equal To (<=)

explanation: used the where clause to display employees whose salary is less than or equal to rs.40,000.



The screenshot shows a SQL IDE interface. At the top, a tab labeled 'SQL File 3*' is open. Below the tab is a toolbar with various icons for file operations, execution, and search. A dropdown menu shows 'Limit to 1000 rows'. The main text area contains the following SQL query:

```
1 select emp_name,salary from collage_db.employees where salary <= 40000;
```

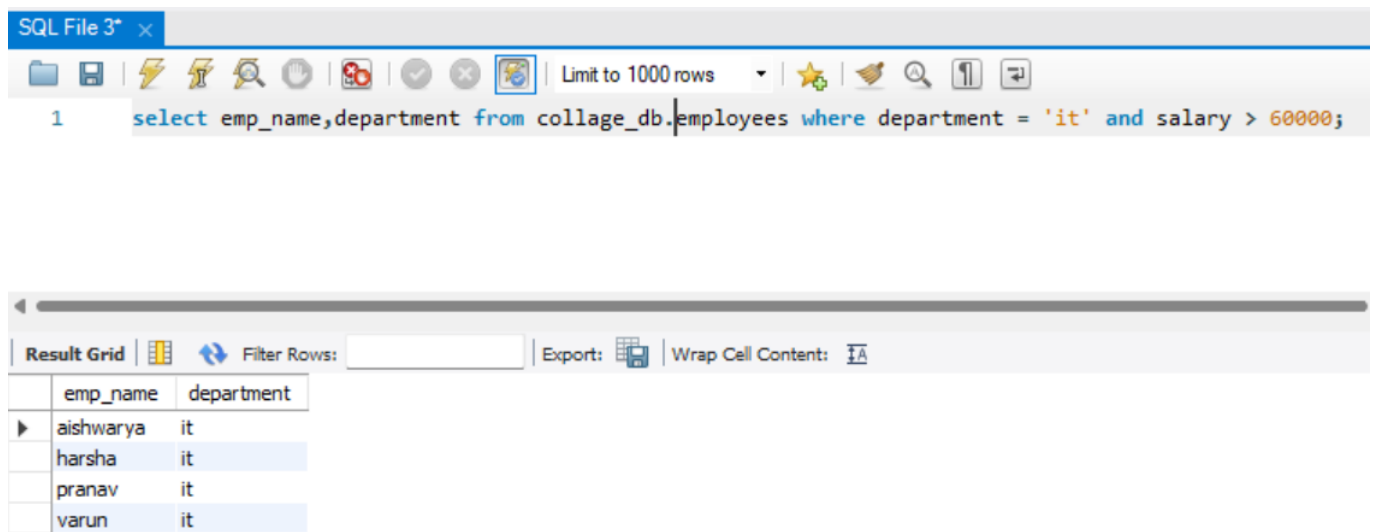
Below the query editor, there is a 'Result Grid' section. It includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The result grid displays the following data:

	emp_name	salary
▶	nani	20000.00
	raj	30000.00
	arjun	35000.00
	nag	26000.00
	harika	40000.00

Stage 5 – Logical Operators

Using AND

explanation: used the and operator to display employees who belong to the it department and have a salary greater than rs.60,000.



The screenshot shows a SQL File 3* window with a toolbar and a query editor. The query is: `select emp_name,department from collage_db.employees where department = 'it' and salary > 60000;`. Below the query editor, the results are displayed in a table with columns `emp_name` and `department`. The results show four employees: aishwarya, harsha, pranav, and varun, all belonging to the 'it' department.

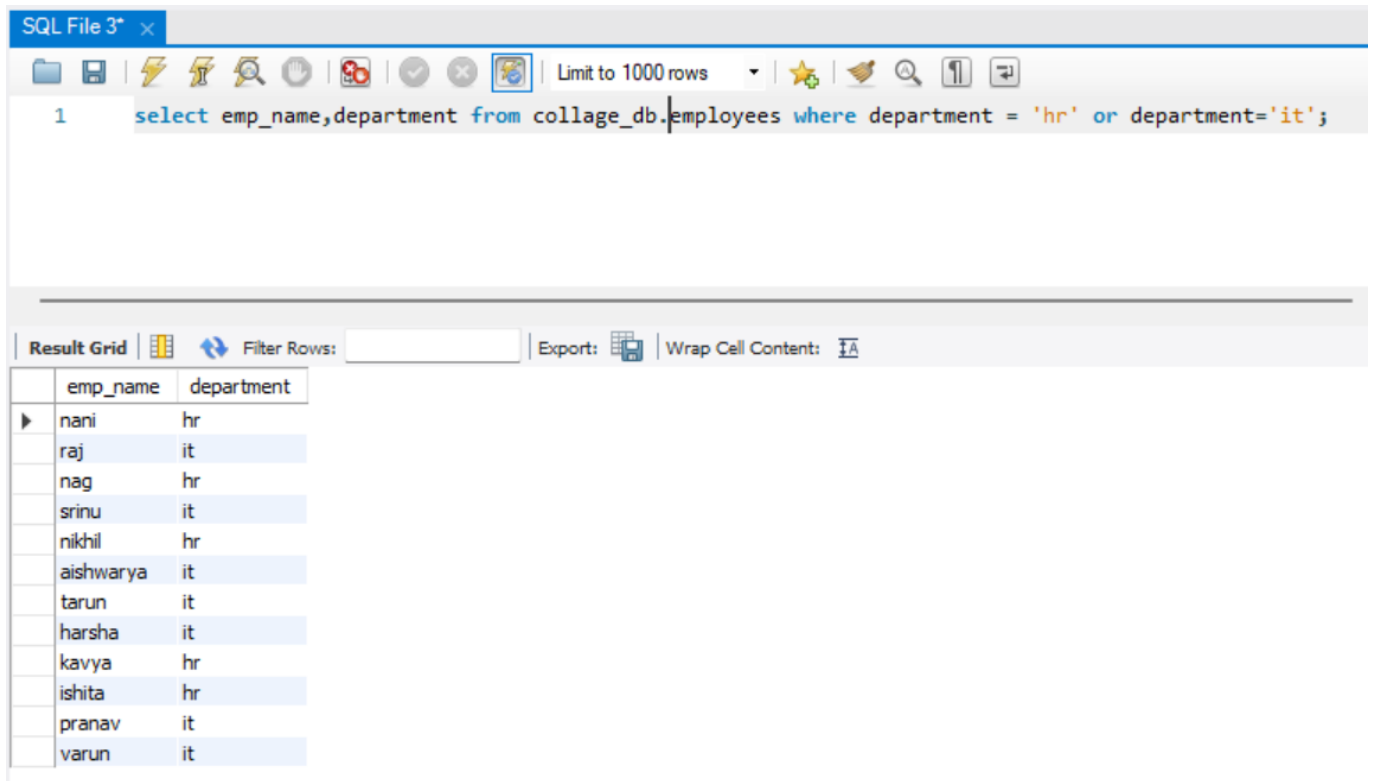
```
1 select emp_name,department from collage_db.employees where department = 'it' and salary > 60000;
```

emp_name	department
aishwarya	it
harsha	it
pranav	it
varun	it

Stage 5 – Logical Operators

Using OR

explanation: used the or operator to display employees who belong to either the hr or it department.



The screenshot shows a SQL IDE window titled "SQL File 3*" with a toolbar and a query editor. The query editor contains the following SQL statement:

```
1 select emp_name,department from collage_db.employees where department = 'hr' or department='it';
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has two columns: "emp_name" and "department". The results are as follows:

emp_name	department
nani	hr
raj	it
nag	hr
srinu	it
nikhil	hr
aishwarya	it
tarun	it
harsha	it
kavya	hr
ishita	hr
pranav	it
varun	it

Stage 5 – Logical Operators

Using NOT

explanation: used the not operator to display employees who are not from the hr department.

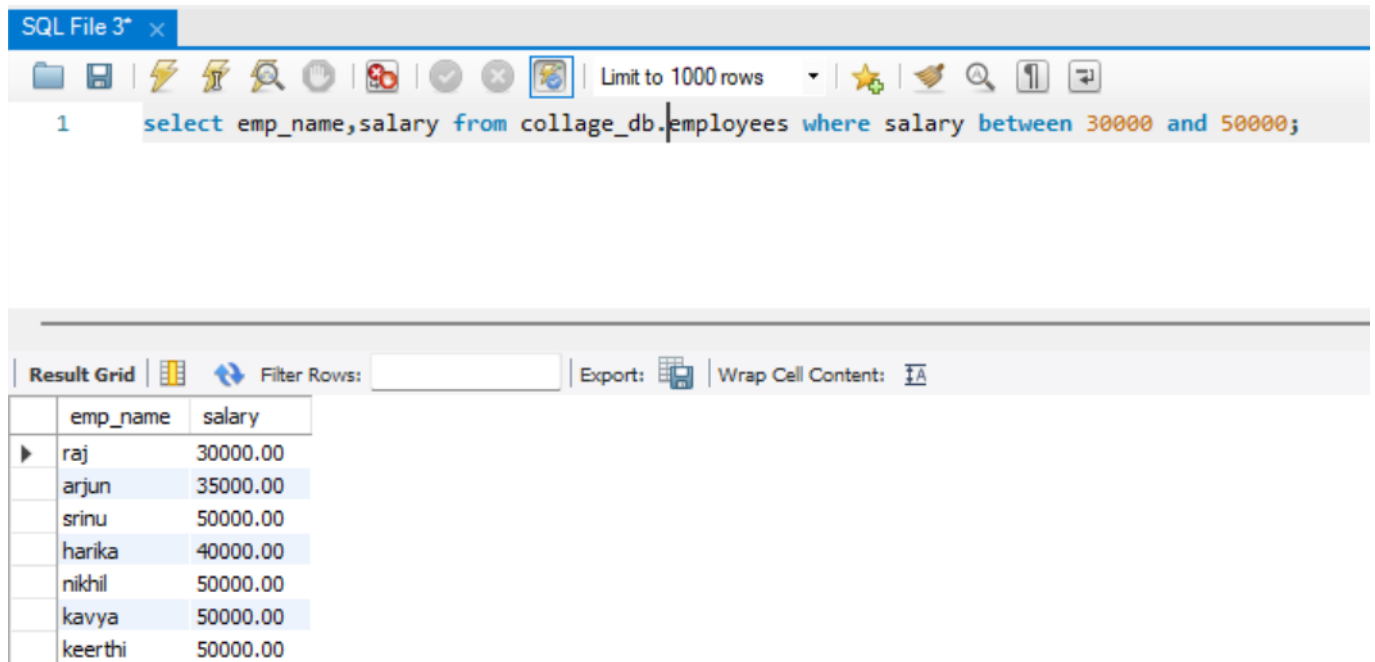
```
SQL File 3* x
Limit to 1000 rows
1 select emp_name,department from collage_db.employees where not department = 'hr';
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	emp_name	department			
▶	raj	it			
	arjun	finance			
	srinu	it			
	harika	finance			
	faizan	marketing			
	aishwarya	it			
	tarun	it			
	harsha	it			
	abhishek	finance			
	sanjana	marketing			
	rakesh	finance			
	keerthi	marketing			
	pranav	it			
	madhuri	marketing			
	varun	it			

Stage 6 – Special Operators

Using BETWEEN for Salary

explanation: used between to display employees whose salary is between rs.30,000 and rs.50,000.

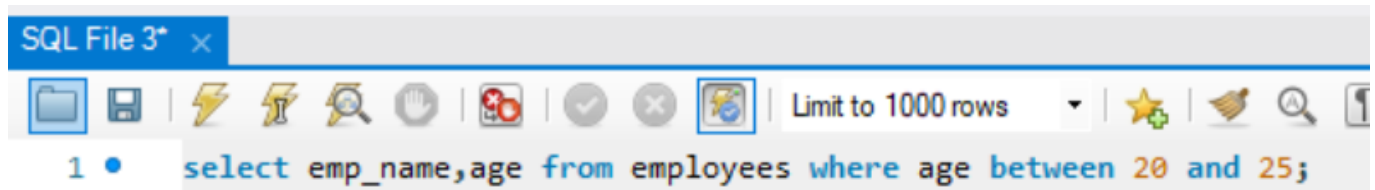


The screenshot shows a SQL IDE interface. At the top, a tab labeled "SQL File 3*" is open. Below the tab is a toolbar with various icons, including a "Limit to 1000 rows" dropdown. The main area displays a SQL query: `1 select emp_name,salary from collage_db.employees where salary between 30000 and 50000;`. Below the query editor, the "Result Grid" is visible, showing the results of the query. The grid has two columns: "emp_name" and "salary". The results are as follows:

emp_name	salary
raj	30000.00
arjun	35000.00
srinu	50000.00
harika	40000.00
nikhil	50000.00
kavya	50000.00
keerthi	50000.00

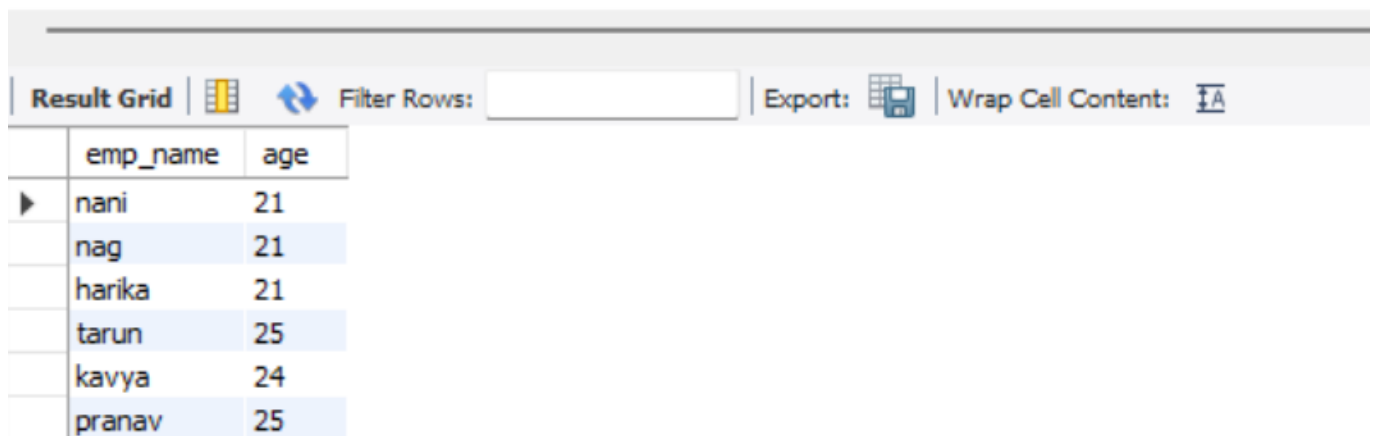
Using BETWEEN for Age

explanation: used between to display employees whose age is between 20 and 25 years.



The screenshot shows a SQL editor window titled "SQL File 3*". The toolbar includes icons for file operations, execution, and a "Limit to 1000 rows" dropdown. The SQL query entered is:

```
1 • select emp_name,age from employees where age between 20 and 25;
```



The screenshot shows the "Result Grid" of a database application. It displays the results of the SQL query in a table with two columns: "emp_name" and "age". The results are as follows:

	emp_name	age
▶	nani	21
	nag	21
	harika	21
	tarun	25
	kavya	24
	pranav	25

Stage 6 – Special Operators

Using IN for Department

explanation: used in to display employees who belong to either the hr or it department.

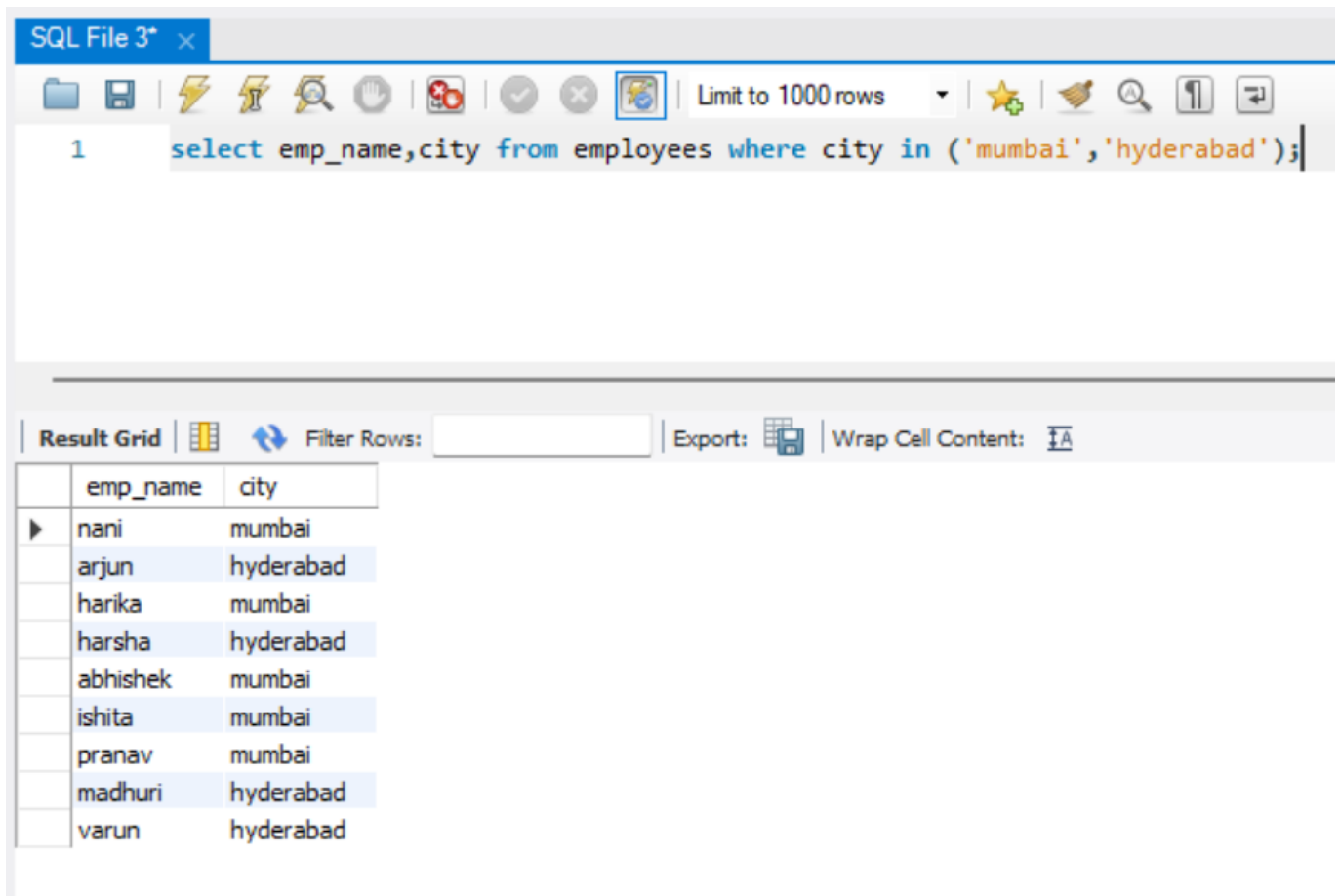
```
SQL File 3* x
1 select emp_name,department from employees where department in ('hr','it');
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	emp_name	department			
▶	nani	hr			
	raj	it			
	nag	hr			
	srinu	it			
	nikhil	hr			
	aishwarya	it			
	tarun	it			
	harsha	it			
	kavya	hr			
	ishita	hr			
	pranav	it			
	varun	it			

Stage 6 – Special Operators

Using IN for City

explanation: used in to display employees who are from either mumbai or hyderabad.



The screenshot shows an SQL IDE interface. The top toolbar includes icons for file operations, execution, and settings. The query editor displays the following SQL statement:

```
1 select emp_name,city from employees where city in ('mumbai','hyderabad');
```

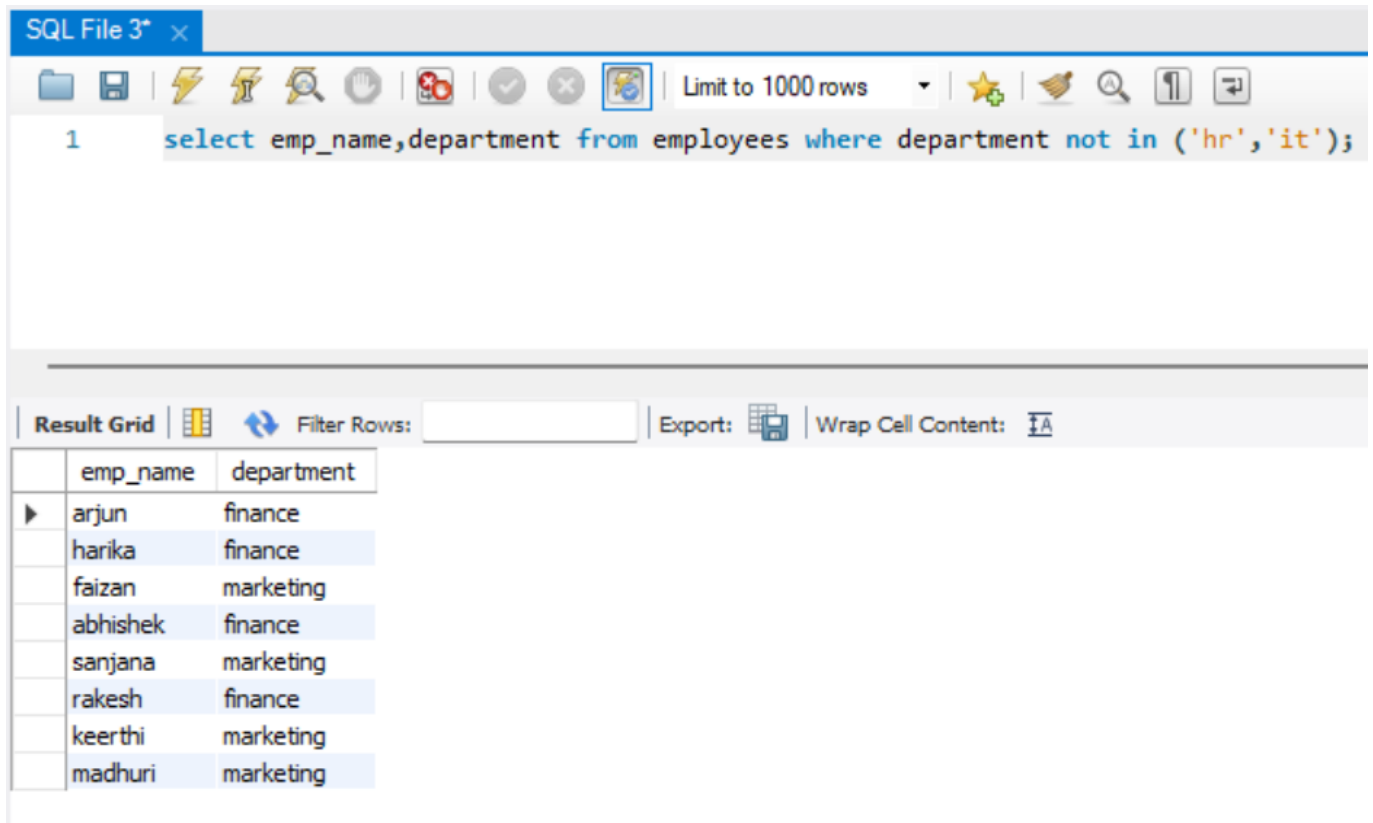
Below the query editor, the 'Result Grid' tab is active, showing the results of the query. The results are displayed in a table with two columns: 'emp_name' and 'city'.

emp_name	city
nani	mumbai
arjun	hyderabad
harika	mumbai
harsha	hyderabad
abhishek	mumbai
ishita	mumbai
pranav	mumbai
madhuri	hyderabad
varun	hyderabad

Stage 6 – Special Operators

Using NOT IN for Department

explanation: used not in to display employees who are not from the hr or it departments.



The screenshot shows a SQL IDE window titled "SQL File 3*". The query editor contains the following SQL statement:

```
1 select emp_name,department from employees where department not in ('hr','it');
```

Below the query editor, the "Result Grid" displays the output of the query. The grid has two columns: "emp_name" and "department". The results are as follows:

emp_name	department
arjun	finance
harika	finance
faizan	marketing
abhishek	finance
sanjana	marketing
rakesh	finance
keerthi	marketing
madhuri	marketing

Stage 6 – Special Operators

Using NOT IN for City

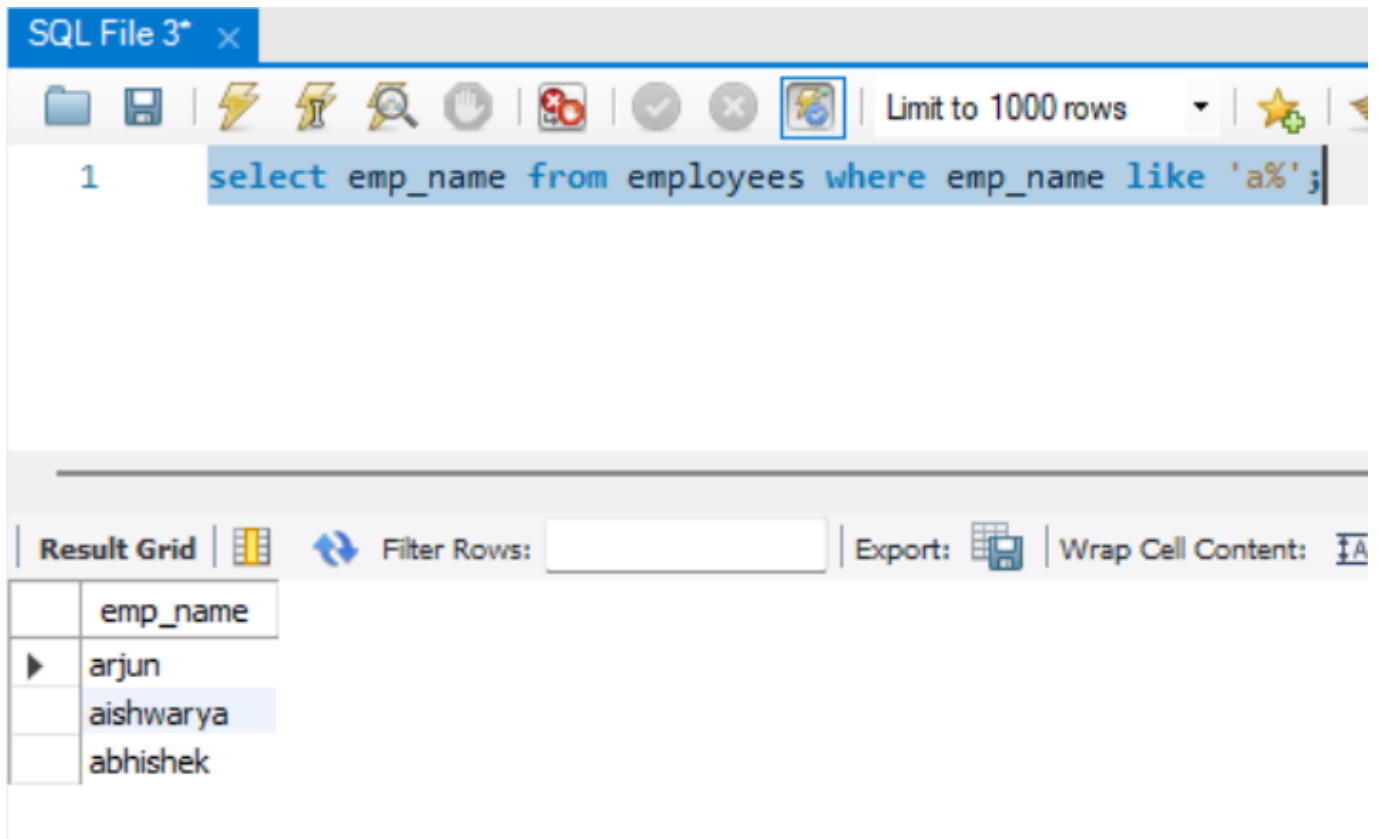
explanation: used not in to display employees who are not from mumbai or hyderabad.

```
SQL File 3* x
Limit to 1000 rows
1 select emp_name,city from employees where city not in ('mumbai','hyderabad');
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	emp_name	city			
▶	raj	delhi			
	nag	bombay			
	srinu	delhi			
	faizan	bengaluru			
	nikhil	bengaluru			
	aishwarya	chennai			
	kavya	bengaluru			
	sanjana	chennai			
	rakesh	bengaluru			
	keerthi	chennai			

Using LIKE

explanation: used like with the pattern a% to display employee names that start with the letter a.

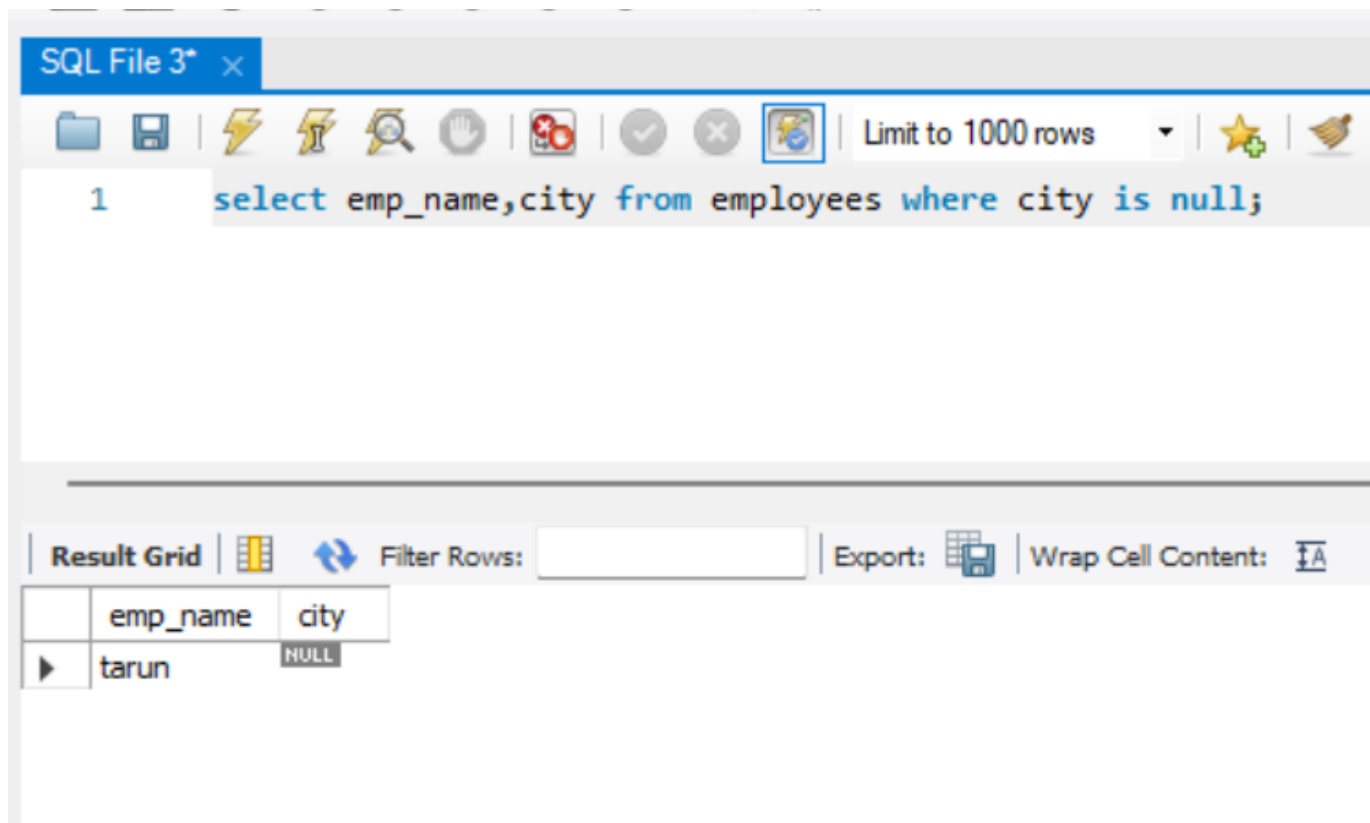


The screenshot shows a SQL IDE interface. At the top, a tab labeled "SQL File 3*" is active. Below the tab is a toolbar with various icons, including a folder, save, undo, redo, search, and a "Limit to 1000 rows" dropdown. The main editor area contains a single line of SQL code: `1 select emp_name from employees where emp_name like 'a%';`. Below the editor is a "Result Grid" section. It includes a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. The result grid itself is a table with one column, "emp_name", and three rows of data: "arjun", "aishwarya", and "abhishek". The row containing "aishwarya" is highlighted in blue.

emp_name
arjun
aishwarya
abhishek

Checking IS NULL

explanation: used is null to display employees whose city value is not provided.



The screenshot shows an SQL IDE interface. At the top, a tab labeled "SQL File 3*" is open. Below the tab is a toolbar with various icons, including a folder, save, lightning bolt, magnifying glass, hand, and a button with a lightning bolt and a circle, which is highlighted with a blue box. To the right of the toolbar is a dropdown menu set to "Limit to 1000 rows". Below the toolbar, the SQL query is entered in a text area:

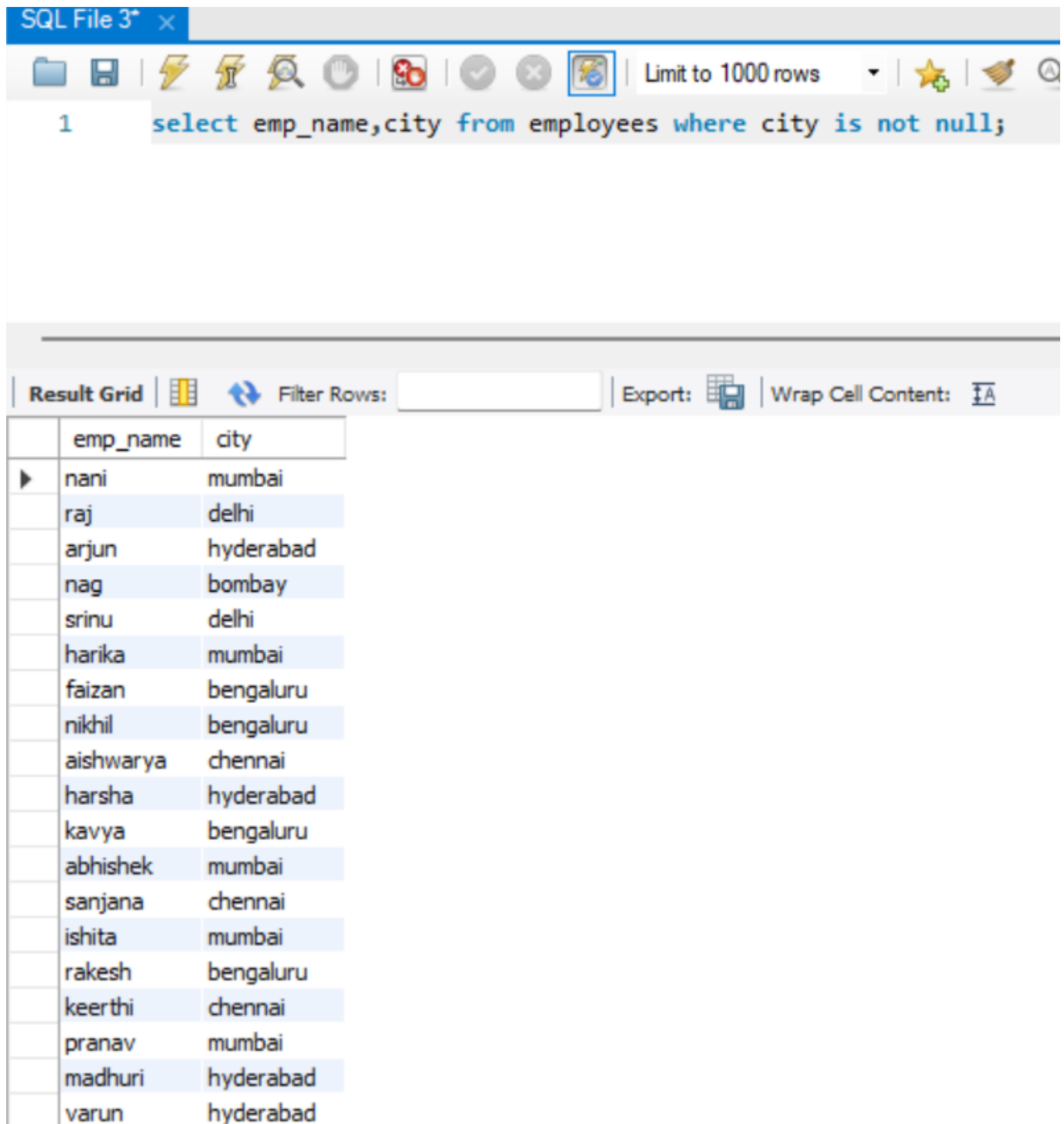
```
1 select emp_name,city from employees where city is null;
```

Below the query area, there is a section for the results. It includes a "Result Grid" button, a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" button. Below these buttons is a table with the following data:

	emp_name	city
▶	tarun	NULL

Checking IS NOT NULL

explanation: used is not null to display employees whose city value is provided.



The screenshot shows a SQL IDE window titled "SQL File 3*". The query editor contains the following SQL statement:

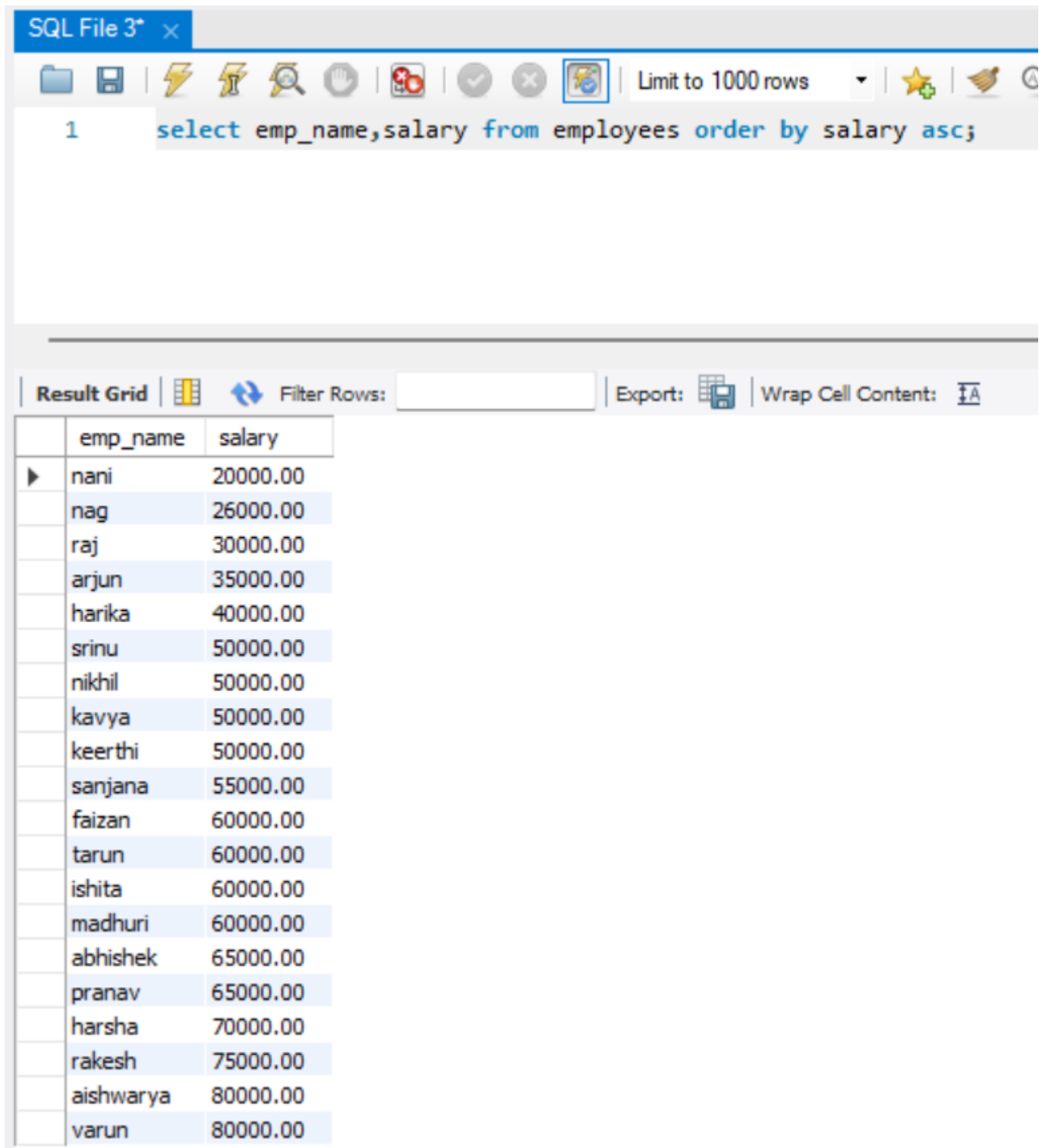
```
1 select emp_name,city from employees where city is not null;
```

Below the query editor, the "Result Grid" tab is active, displaying the results of the query. The results are shown in a table with two columns: "emp_name" and "city". The table contains 20 rows of data, listing employee names and their corresponding cities.

emp_name	city
nani	mumbai
raj	delhi
arjun	hyderabad
nag	bombay
srinu	delhi
harika	mumbai
faizan	bengaluru
nikhil	bengaluru
aishwarya	chennai
harsha	hyderabad
kavya	bengaluru
abhishek	mumbai
sanjana	chennai
ishita	mumbai
rakesh	bengaluru
keerthi	chennai
pranav	mumbai
madhuri	hyderabad
varun	hyderabad

Sorting Salary in Ascending Order

explanation: used order by with asc to display employee names and salaries in ascending order of salary.



The screenshot shows a SQL IDE window titled "SQL File 3* x". The query editor contains the following SQL statement:

```
1 select emp_name,salary from employees order by salary asc;
```

The results are displayed in a "Result Grid" below the query editor. The grid has two columns: "emp_name" and "salary". The results are sorted in ascending order of salary.

	emp_name	salary
▶	nani	20000.00
	nag	26000.00
	raj	30000.00
	arjun	35000.00
	harika	40000.00
	srinu	50000.00
	nikhil	50000.00
	kavya	50000.00
	keerthi	50000.00
	sanjana	55000.00
	faizan	60000.00
	tarun	60000.00
	ishita	60000.00
	madhuri	60000.00
	abhishek	65000.00
	pranav	65000.00
	harsha	70000.00
	rakesh	75000.00
	aishwarya	80000.00
	varun	80000.00

Stage 7 – ORDER BY

Sorting Salary in Descending Order

explanation: used order by with desc to display employee names and salaries in descending order of salary.

```
SQL File 3* x
[Icons: Folder, Save, Run, Undo, Redo, Find, Stop, Refresh, Close, Limit to 1000 rows, Star, Erase, Search]
1 select emp_name,salary from employees order by salary desc;
```

Result Grid [Grid Icon] [Refresh Icon] Filter Rows: Export: [Export Icon] Wrap Cell Content: [Wrap Icon]		
	emp_name	salary
▶	aishwarya	80000.00
	varun	80000.00
	rakesh	75000.00
	harsha	70000.00
	abhishek	65000.00
	pranav	65000.00
	faizan	60000.00
	tarun	60000.00
	ishita	60000.00
	madhuri	60000.00
	sanjana	55000.00
	srinu	50000.00
	nikhil	50000.00
	kavya	50000.00
	keerthi	50000.00
	harika	40000.00
	arjun	35000.00
	raj	30000.00
	nag	26000.00
	nani	20000.00

Sorting by Age

explanation: used order by with asc to display employee names and ages in ascending order of age.

```
SQL File 3* x
1 select emp_name,age from employees order by age asc;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	emp_name	age			
▶	srinu	19			
	nani	21			
	nag	21			
	harika	21			
	kavya	24			
	tarun	25			
	pranav	25			
	arjun	26			
	sanjana	26			
	harsha	27			
	raj	28			
	ishita	28			
	nikhil	29			
	keerthi	29			
	abhishek	30			
	madhuri	31			
	varun	32			
	faizan	35			
	rakesh	35			
	aishwarya	37			

Stage 7 – ORDER BY

Sorting by Department

explanation: used order by with asc to display employee names and departments in alphabetical order of department.

```
SQL File 3* x
1 select emp_name,department from employees order by department asc;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	emp_name	department			
▶	arjun	finance			
	harika	finance			
	abhishek	finance			
	rakesh	finance			
	nani	hr			
	nag	hr			
	nikhil	hr			
	kavya	hr			
	ishita	hr			
	raj	it			
	srinu	it			
	aishwarya	it			
	tarun	it			
	harsha	it			
	pranav	it			
	varun	it			
	faizan	marketing			
	sanjana	marketing			
	keerthi	marketing			
	madhuri	marketing			

Stage 7 – ORDER BY

Sorting by Multiple Columns

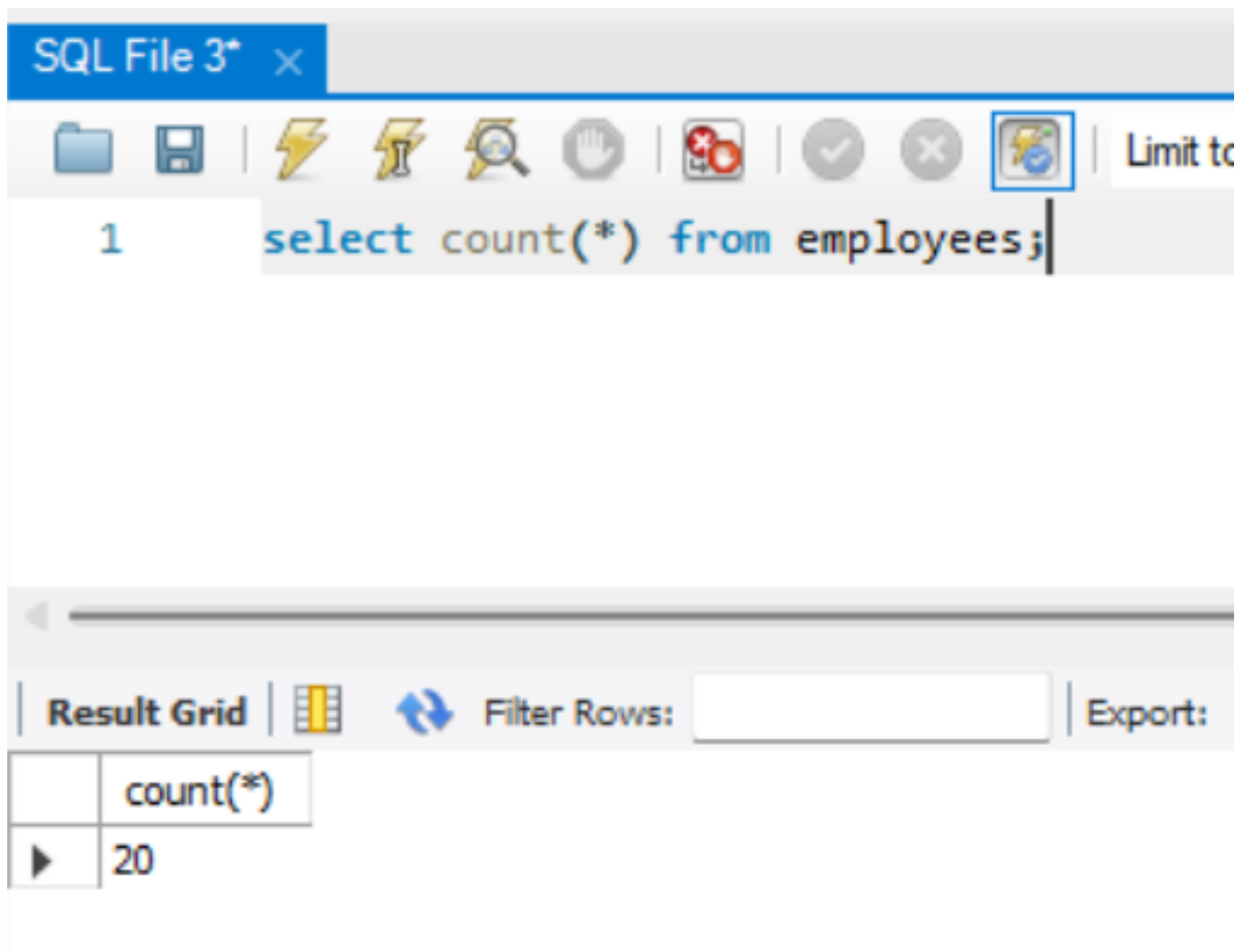
explanation: used order by to sort employee names in ascending order and salary in descending order as the secondary sorting column.

```
SQL File 3* x
Limit to 1000 rows
1 select emp_name,salary from employees order by emp_name asc,salary desc;
```

emp_name	salary
abhishek	65000.00
aishwarya	80000.00
arjun	35000.00
faizan	60000.00
harika	40000.00
harsha	70000.00
ishita	60000.00
kavya	50000.00
keerthi	50000.00
madhuri	60000.00
nag	26000.00
nani	20000.00
nikhil	50000.00
pranav	65000.00
raj	30000.00
rakesh	75000.00
sanjana	55000.00
srinu	50000.00
tarun	60000.00
varun	80000.00

COUNT() – Total Employees

explanation: used count() to calculate the total number of employee records. the table contains 20 employees.



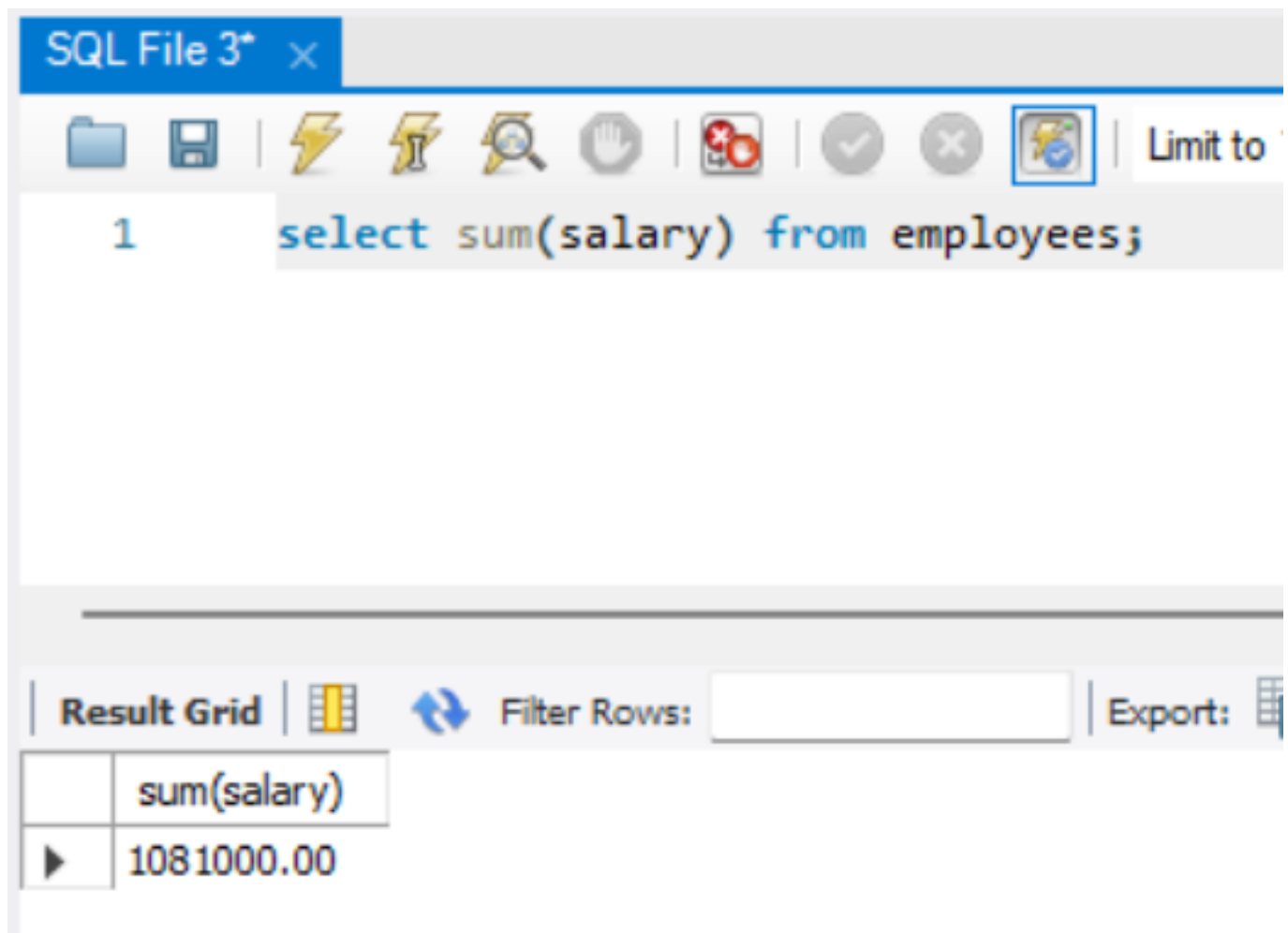
The screenshot shows a SQL IDE window titled "SQL File 3* x". The toolbar includes icons for file operations, execution, and a "Limit to" dropdown. The SQL editor contains the query: `1 select count(*) from employees;`. Below the editor, the "Result Grid" is displayed with a "Filter Rows" input and an "Export" button. The result grid shows a single row with the column name "count(*)" and the value "20".

	count(*)
▶	20

Stage 8 – Aggregate Functions

SUM() – Total Salary

explanation: used sum() to calculate the total salary paid to all employees. the result is rs.10,81,000.



The screenshot shows a SQL IDE window titled "SQL File 3* x". The toolbar includes icons for file operations, execution, search, and a "Limit to" dropdown. The query editor contains the following SQL statement:

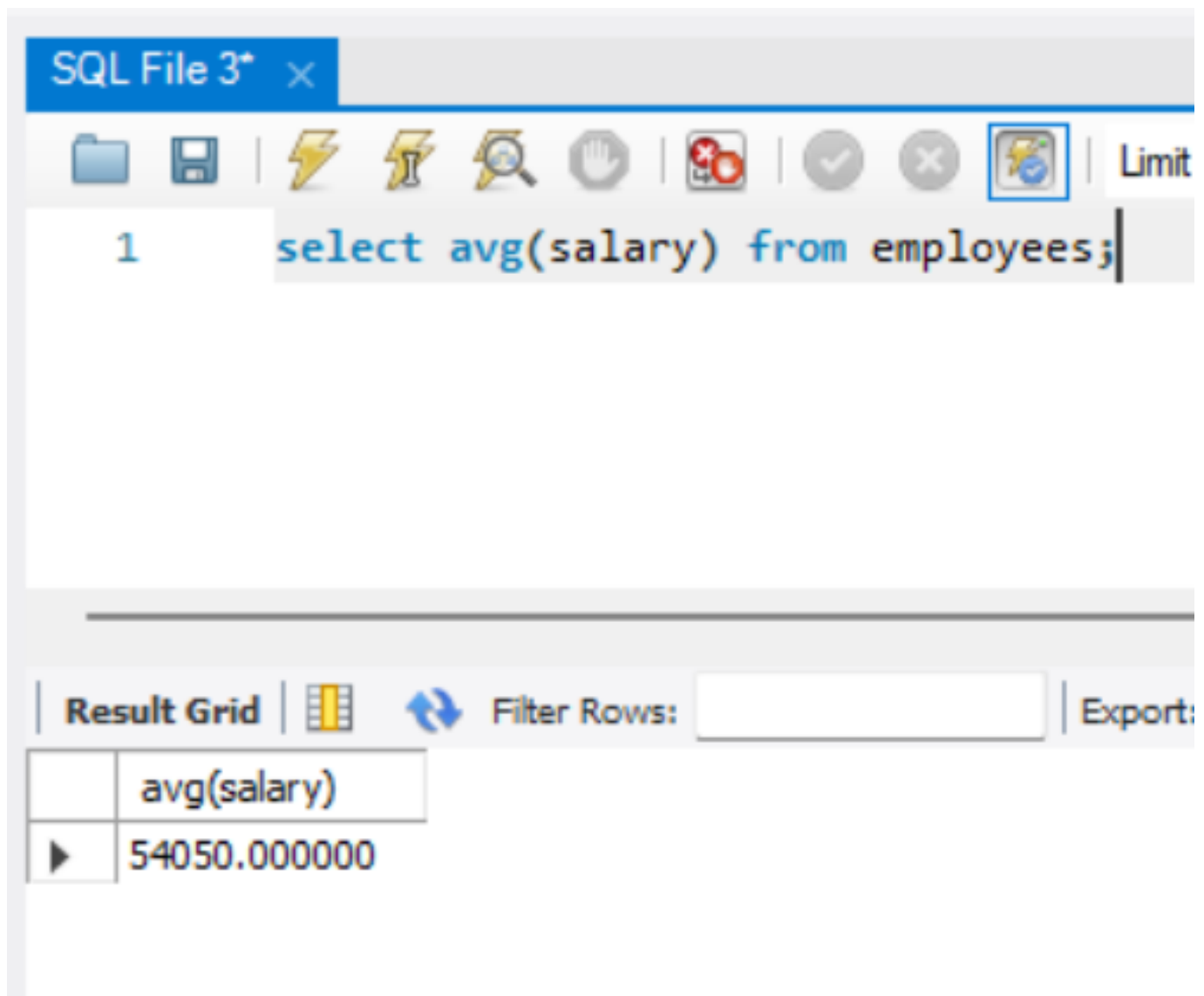
```
1 select sum(salary) from employees;
```

Below the query editor, the "Result Grid" is displayed. It shows a single column header "sum(salary)" and one data row with the value "1081000.00".

	sum(salary)
▶	1081000.00

AVG() – Average Salary

explanation: used avg() to calculate the average salary of all employees. the result is rs.54,050.



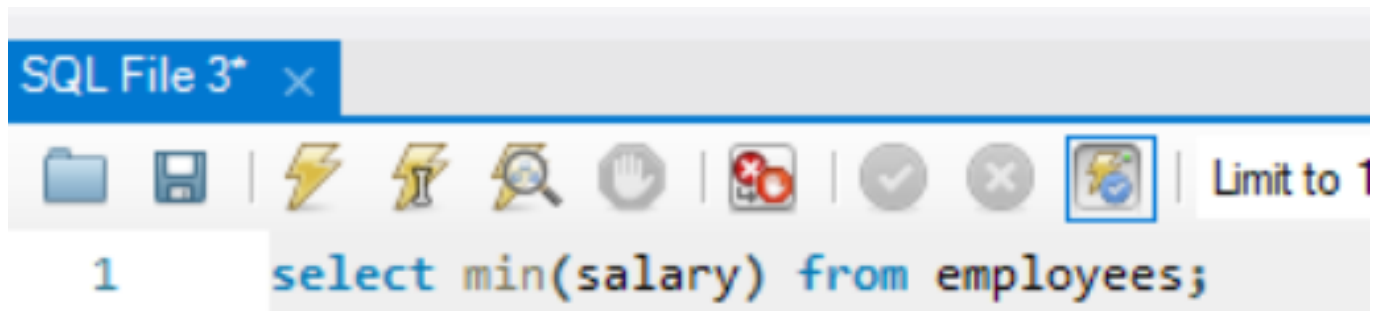
The screenshot shows a SQL IDE window titled "SQL File 3* x". The toolbar includes icons for file operations, execution, search, and window management. The SQL editor contains the query: `1 select avg(salary) from employees;`. Below the editor, the "Result Grid" tab is active, displaying the query result in a table.

	avg(salary)
▶	54050.000000

Stage 8 – Aggregate Functions

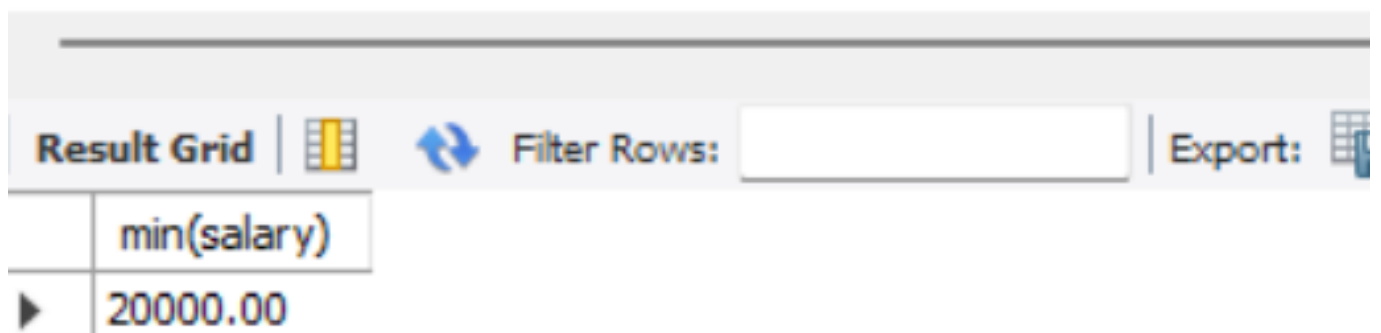
MIN() – Lowest Salary

explanation: used min() to find the lowest salary among all employees. the result is rs.20,000.



SQL File 3* x

1 select min(salary) from employees;



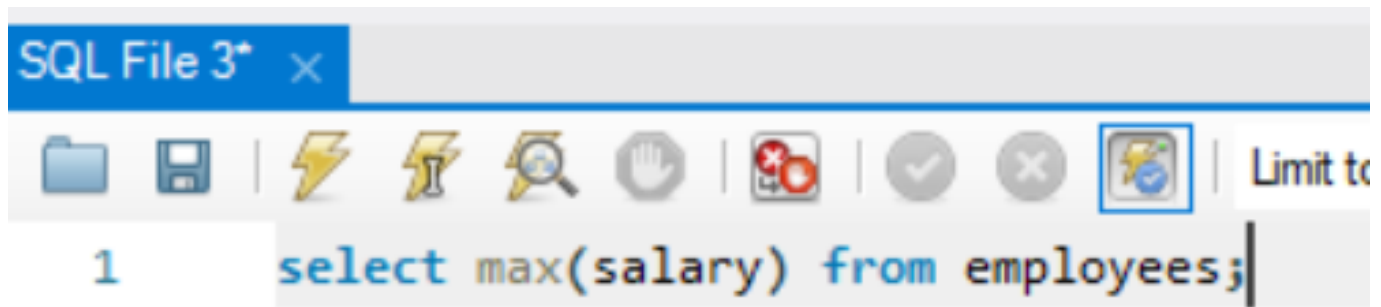
Result Grid | Filter Rows: | Export:

	min(salary)
▶	20000.00

Stage 8 – Aggregate Functions

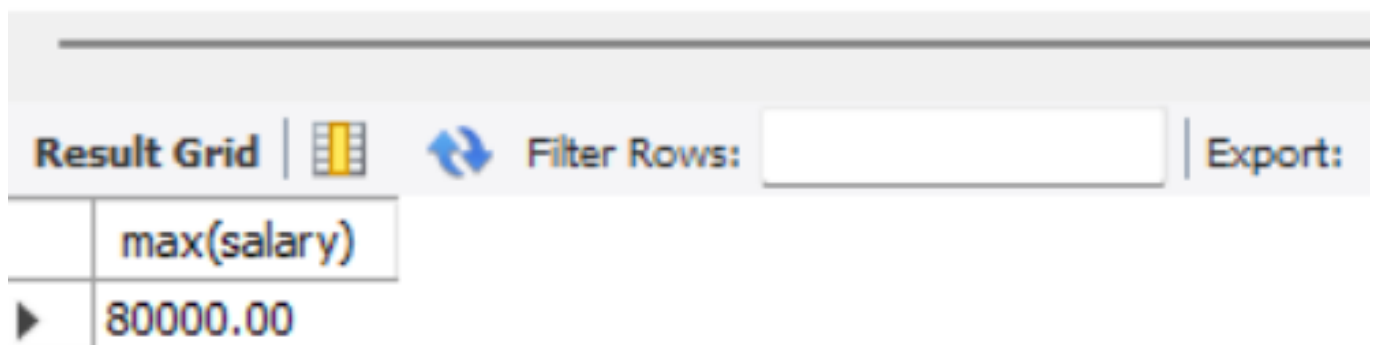
MAX() – Highest Salary

explanation: used max() to find the highest salary among all employees. the result is rs.80,000.



The screenshot shows an SQL IDE window titled "SQL File 3*". The toolbar includes icons for file operations (folder, save), execution (lightning bolt, pin), search (magnifying glass), and window management (stop, refresh, close, maximize). The SQL editor contains the following query:

```
1 select max(salary) from employees;
```



The screenshot shows the "Result Grid" tab in the SQL IDE. The toolbar includes a "Filter Rows" button and an "Export" button. The result grid displays the output of the query:

	max(salary)
▶	80000.00

Stage 9 – GROUP BY

Department-wise Employee Count

explanation: used group by with count() to find the total number of employees in each department.

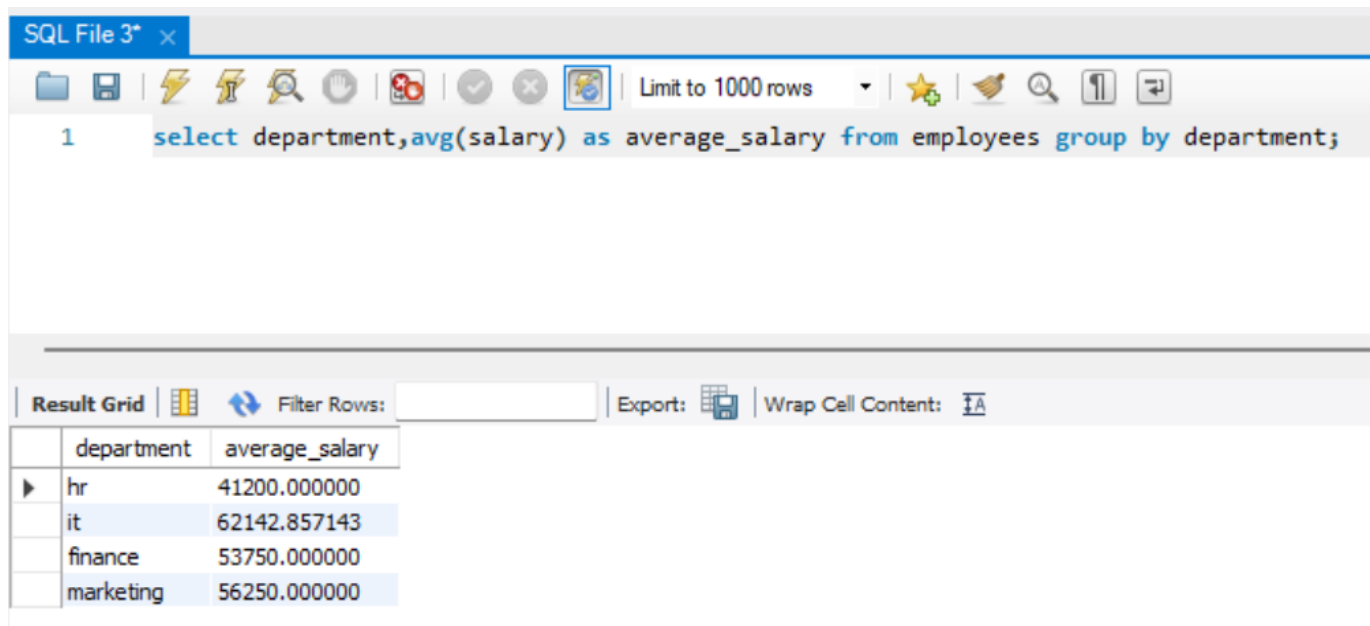
```
SQL File 3* x
1 select department, count(*) as total_employees from employees group by department;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	department	total_employees			
▶	hr	5			
	it	7			
	finance	4			
	marketing	4			

Stage 9 – GROUP BY

Department-wise Average Salary

explanation: used group by with avg() to calculate the average salary for each department.



The screenshot shows a SQL IDE interface. At the top, there's a tab labeled "SQL File 3* x". Below the tab is a toolbar with various icons for file operations, execution, and editing. A dropdown menu shows "Limit to 1000 rows". The main area contains a single SQL query:

```
1 select department, avg(salary) as average_salary from employees group by department;
```

Below the query editor, there's a "Result Grid" section. It includes a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. The result grid displays the following data:

	department	average_salary
▶	hr	41200.000000
	it	62142.857143
	finance	53750.000000
	marketing	56250.000000

Stage 9 – GROUP BY

Department-wise Minimum Salary

explanation: used group by with min() to find the minimum salary in each department.

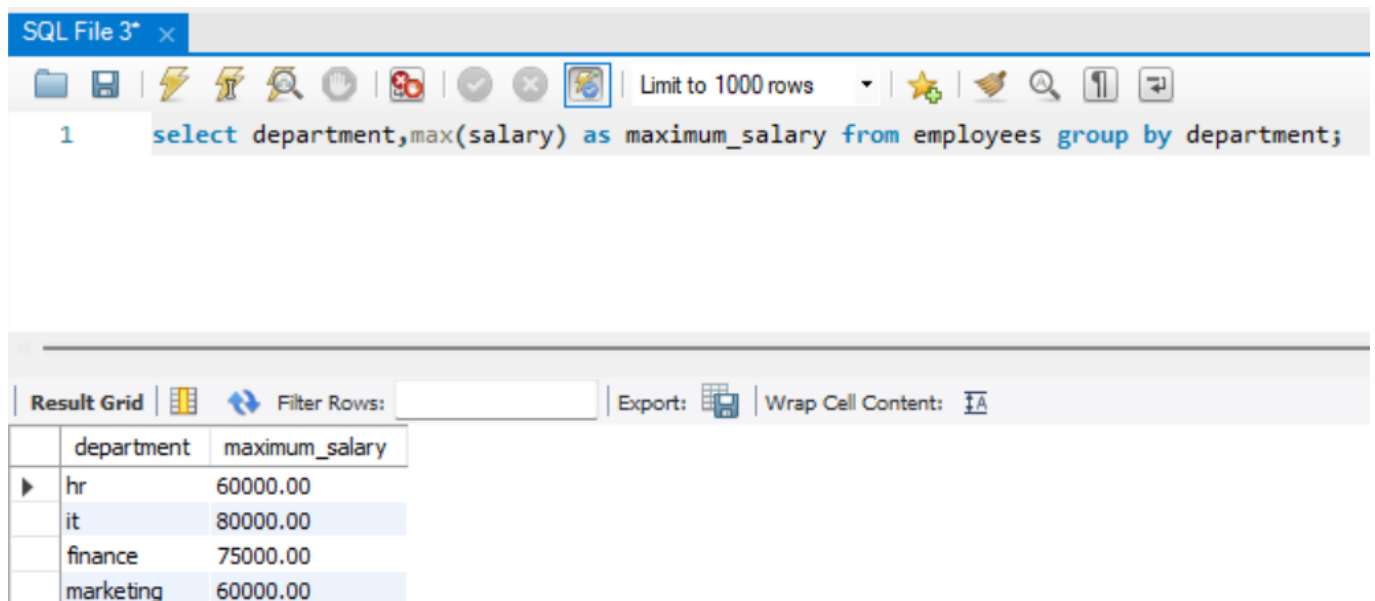
```
SQL File 3* x
1 select department,min(salary) as minimum_salary from employees group by department;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	department	minimum_salary			
▶	hr	20000.00			
	it	30000.00			
	finance	35000.00			
	marketing	50000.00			

Stage 9 – GROUP BY

Department-wise Maximum Salary

explanation: used group by with max() to find the maximum salary in each department.



The screenshot shows an SQL IDE interface. At the top, there's a tab labeled "SQL File 3*". Below the tab is a toolbar with various icons for file operations, execution, and viewing. A dropdown menu shows "Limit to 1000 rows". The main area contains a single SQL query:

```
1 select department,max(salary) as maximum_salary from employees group by department;
```

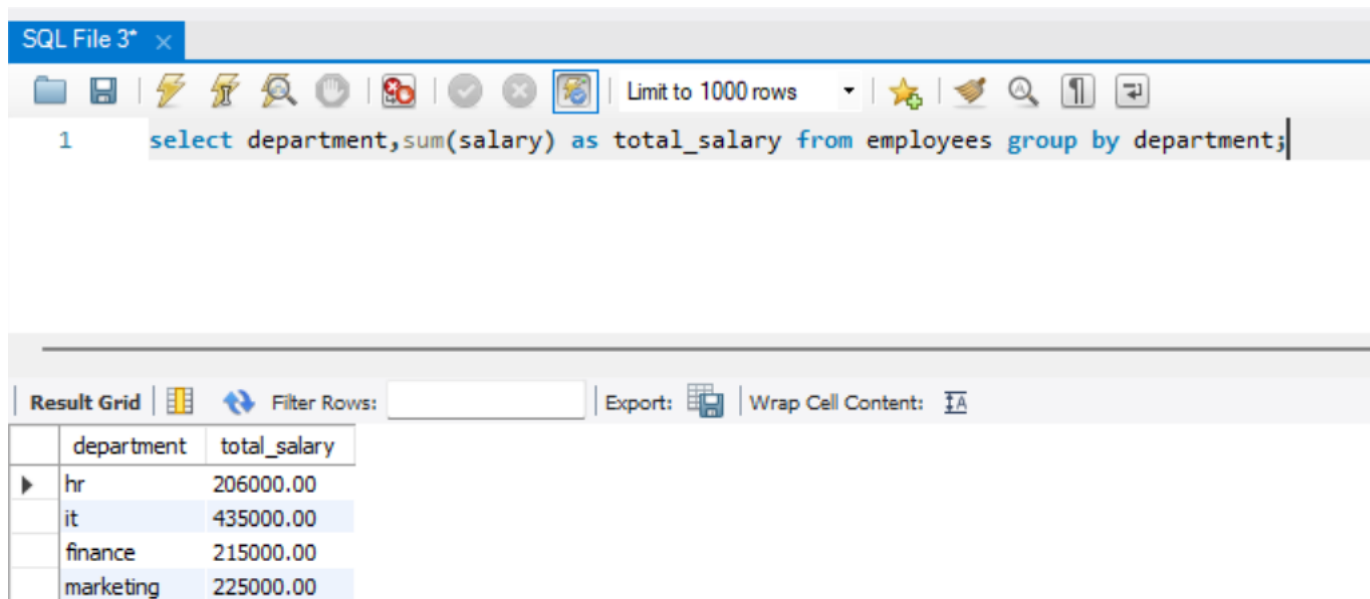
Below the query editor, there's a section for the "Result Grid". It includes a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. The result grid itself displays the following data:

	department	maximum_salary
▶	hr	60000.00
	it	80000.00
	finance	75000.00
	marketing	60000.00

Stage 9 – GROUP BY

Department-wise Total Salary

explanation: used group by with sum() to calculate the total salary for each department.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and settings. The SQL editor contains the following query:

```
1 select department,sum(salary) as total_salary from employees group by department;
```

Below the editor, the 'Result Grid' tab is active, displaying the query results in a table. The table has two columns: 'department' and 'total_salary'. The results are as follows:

department	total_salary
hr	206000.00
it	435000.00
finance	215000.00
marketing	225000.00

City-wise Employee Count

explanation: used group by with count() to find the number of employees in each city.

```
SQL File 3* x
1 select city,count(*) as employees_incity from employees group by city;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	city	employees_incity			
▶	mumbai	5			
	delhi	2			
	hyderabad	4			
	bombay	1			
	bengaluru	4			
	chennai	3			
	NULL	1			

Stage 9 – GROUP BY

City-wise Average Salary

explanation: used group by with avg() to calculate the average salary of employees in each city.

```
SQL File 3* x
Limit to 1000 rows
1 select city,avg(salary) as avg_salaryincity from employees group by city;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	city	avg_salaryincity			
▶	mumbai	50000.000000			
	delhi	40000.000000			
	hyderabad	61250.000000			
	bombay	26000.000000			
	bengaluru	58750.000000			
	chennai	61666.666667			
	NULL	60000.000000			

Stage 9 – GROUP BY

City-wise Maximum Salary

explanation: used group by with max() to find the maximum salary of employees in each city.

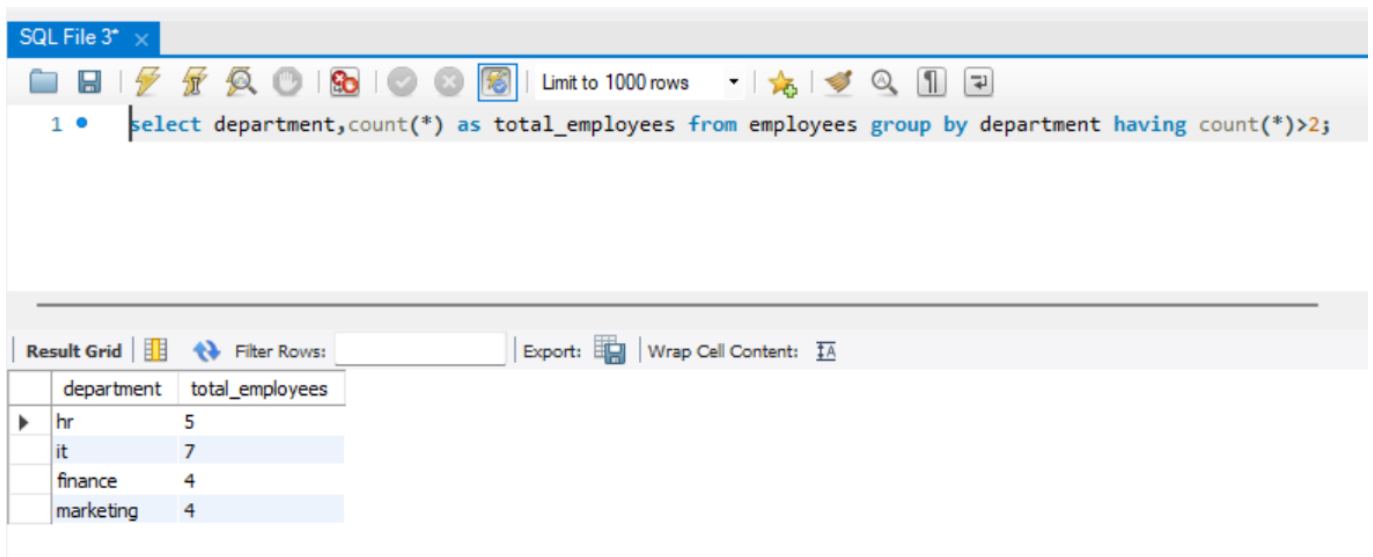
```
SQL File 3* x
1 select city,max(salary) as max_salaryincity from employees group by city;
```

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	city	max_salaryincity			
▶	mumbai	65000.00			
	delhi	50000.00			
	hyderabad	80000.00			
	bombay	26000.00			
	bengaluru	75000.00			
	chennai	80000.00			
	NULL	60000.00			

Stage 10 – HAVING

Departments with More Than 2 Employees

explanation: used group by with having and count() to display departments having more than 2 employees.



The screenshot shows a SQL IDE window titled "SQL File 3*" with a toolbar and a query editor. The query editor contains the following SQL statement:

```
1 • select department, count(*) as total_employees from employees group by department having count(*)>2;
```

Below the query editor, the "Result Grid" tab is active, displaying the results of the query in a table. The table has two columns: "department" and "total_employees". The results are as follows:

department	total_employees
hr	5
it	7
finance	4
marketing	4

Stage 10 – HAVING

Departments with Average Salary Above Rs.20,000

explanation: used group by with having and avg() to display departments whose average salary is greater than rs.20,000.

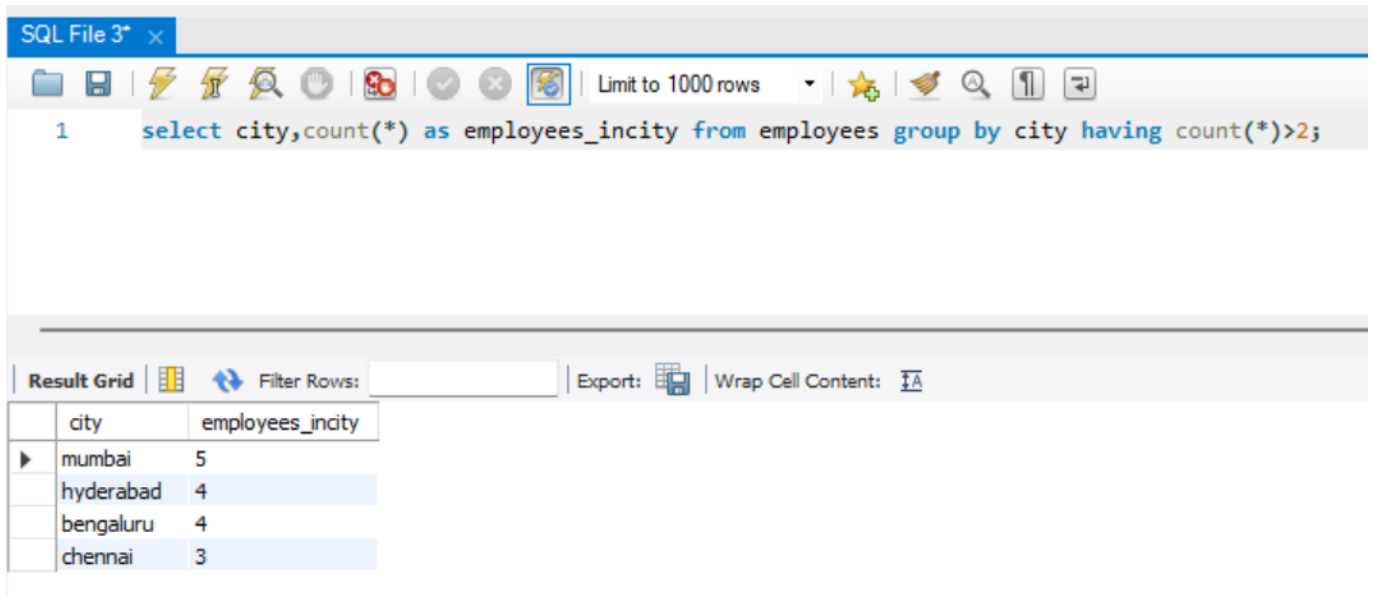
```
SQL File 3* x
[Icons] | Limit to 1000 rows | [Icons]
1 select department,avg(salary) as average_salary from employees group by department having avg(salary)>20000;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	department	average_salary		
▶	hr	41200.000000		
	it	62142.857143		
	finance	53750.000000		
	marketing	56250.000000		

Stage 10 – HAVING

Cities with More Than 2 Employees

explanation: used group by with having and count() to display cities having more than 2 employees.



The screenshot shows a SQL IDE interface. At the top, there's a tab labeled "SQL File 3* x". Below the tab is a toolbar with various icons for file operations, execution, and navigation. A dropdown menu shows "Limit to 1000 rows". The main area contains a SQL query:

```
1 select city,count(*) as employees_incity from employees group by city having count(*)>2;
```

Below the query editor, there's a "Result Grid" section. It includes a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. The result grid displays the following data:

	city	employees_incity
▶	mumbai	5
	hyderabad	4
	bengaluru	4
	chennai	3

Stage 10 – HAVING

Departments with Maximum Salary Above Rs.50,000

explanation: used group by with having and max() to display departments whose maximum salary is greater than rs.50,000.

```
SQL File 3* x
Limit to 1000 rows
1 select department,max(salary) as maximum_salary from employees group by department having max(salary)>50000;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
department	maximum_salary		
hr	60000.00		
it	80000.00		
finance	75000.00		
marketing	60000.00		

Stage 11 – Challenge Questions

Challenge 1 – Second Highest Salary

explanation: used a subquery to find the second-highest salary from the employees table. the result is rs.75,000.

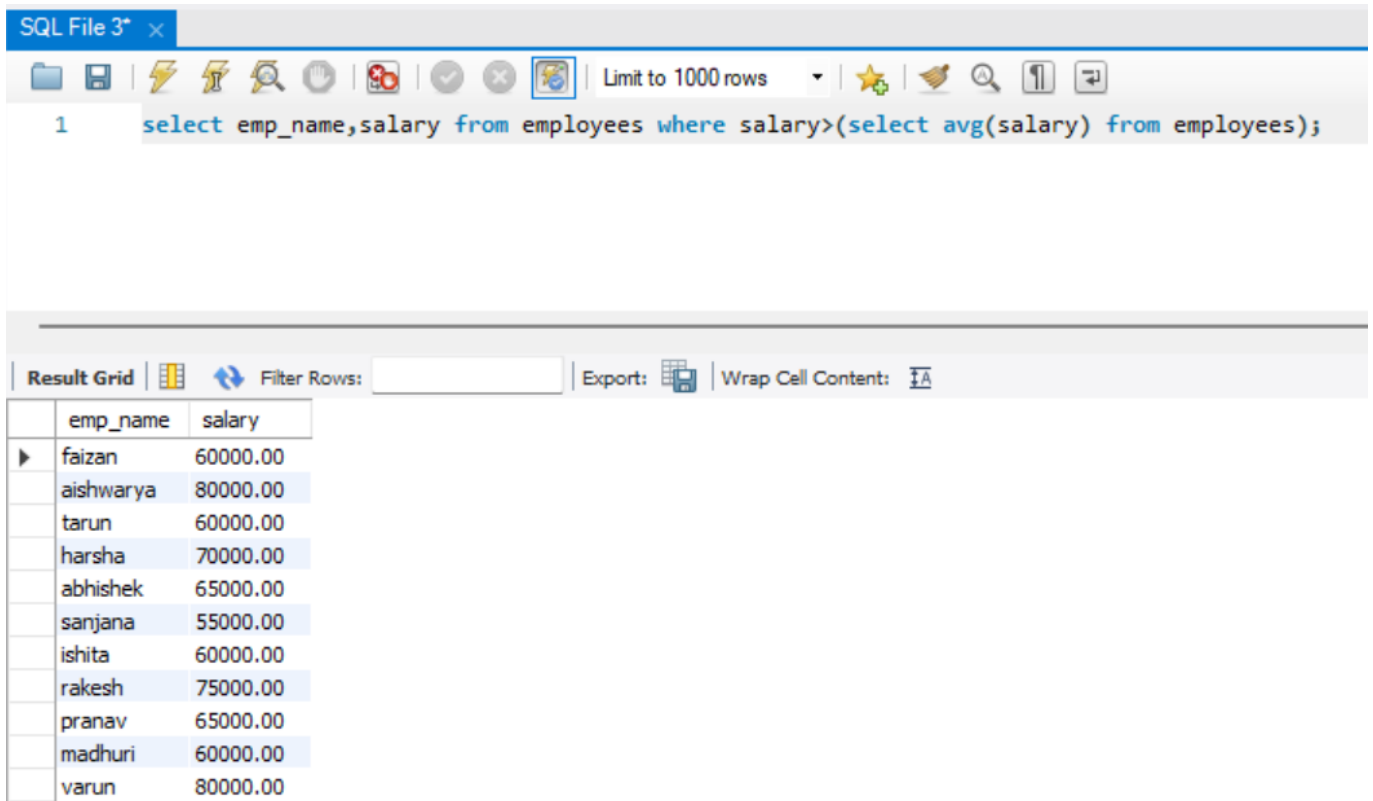
```
SQL File 3* x
Limit to 1000 rows
1 select max(salary) from employees where salary < (select max(salary) from employees);
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
max(salary)			
75000.00			

Stage 11 – Challenge Questions

Challenge 2 – Employees Above Average Salary

explanation: used a subquery with avg() to display employees whose salary is greater than the overall average salary.



The screenshot shows a SQL IDE interface. At the top, a tab labeled 'SQL File 3*' is open. Below the tab is a toolbar with various icons for file operations, execution, and navigation. A dropdown menu shows 'Limit to 1000 rows'. The main text area contains the following SQL query:

```
1 select emp_name,salary from employees where salary>(select avg(salary) from employees);
```

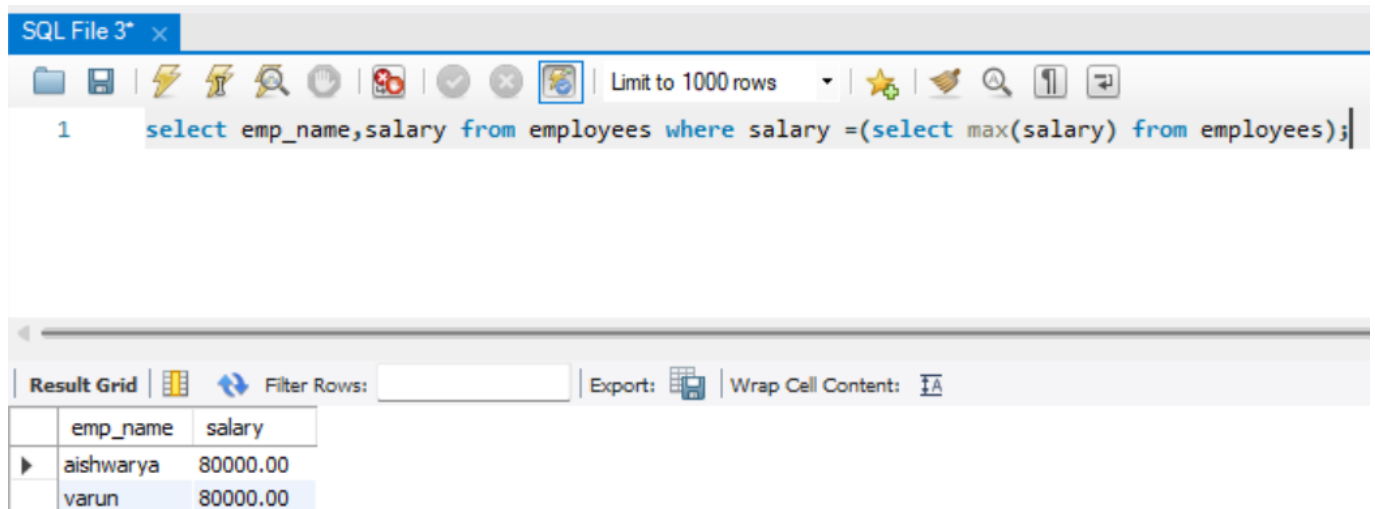
Below the query editor, the 'Result Grid' is displayed. It includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The results are shown in a table with two columns: 'emp_name' and 'salary'.

	emp_name	salary
▶	faizan	60000.00
	aishwarya	80000.00
	tarun	60000.00
	harsha	70000.00
	abhishek	65000.00
	sanjana	55000.00
	ishita	60000.00
	rakesh	75000.00
	pranav	65000.00
	madhuri	60000.00
	varun	80000.00

Stage 11 – Challenge Questions

Challenge 3 – Employee(s) with Highest Salary

explanation: used a subquery with max() to display employees who have the highest salary.



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and settings. The query editor contains the following SQL statement:

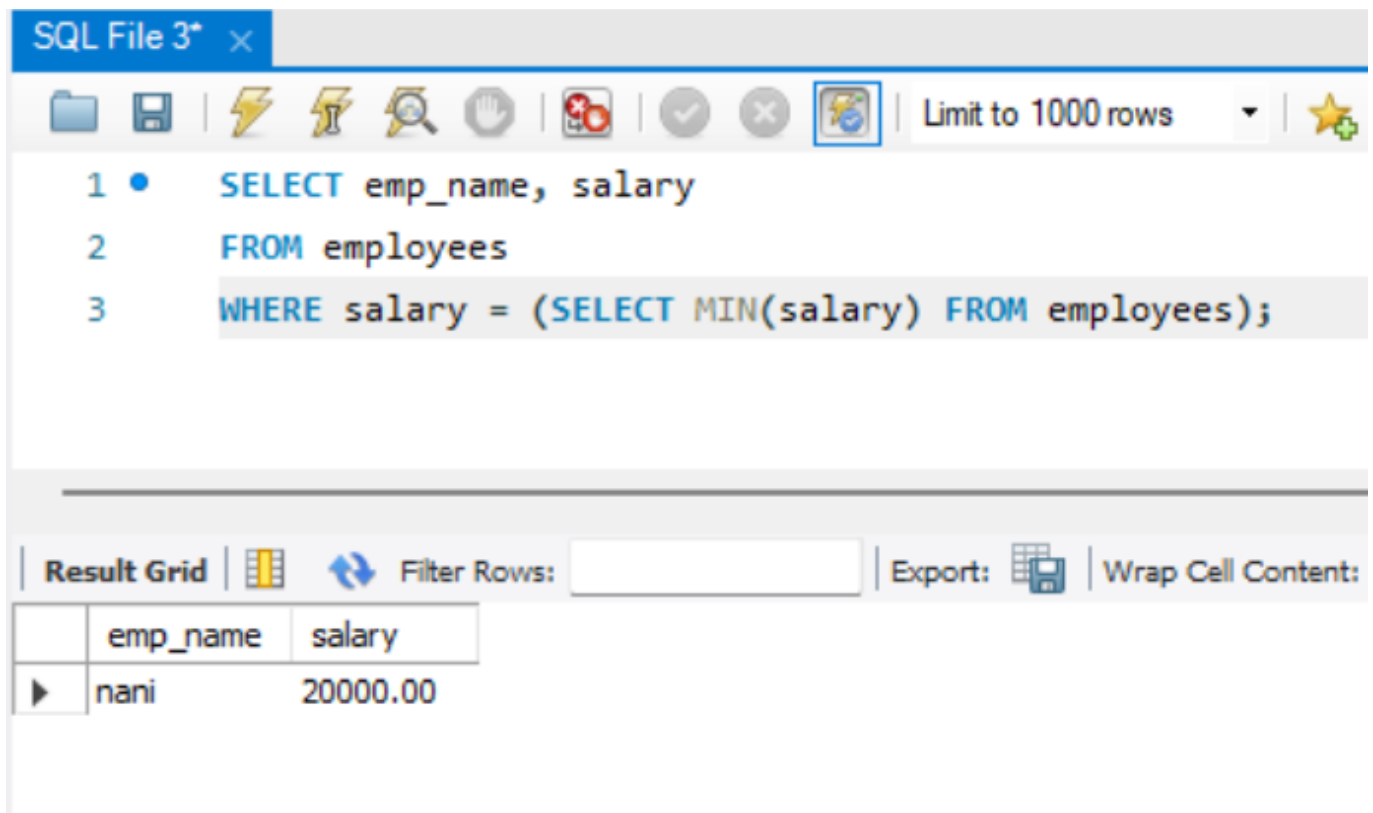
```
1 select emp_name,salary from employees where salary =(select max(salary) from employees);
```

Below the query editor, the 'Result Grid' tab is active, displaying the results of the query in a table format. The table has two columns: 'emp_name' and 'salary'. Two rows are shown, both with a salary of 80000.00.

emp_name	salary
aishwarya	80000.00
varun	80000.00

Challenge 4 – Employee(s) with Lowest Salary

explanation: used a subquery with min() to display employees who have the lowest salary. the result is nani with rs.20,000.



The screenshot shows a SQL IDE window titled "SQL File 3* x". The query editor contains the following SQL code:

```
1 • SELECT emp_name, salary
2 FROM employees
3 WHERE salary = (SELECT MIN(salary) FROM employees);
```

Below the query editor, the "Result Grid" tab is active, displaying the results of the query in a table:

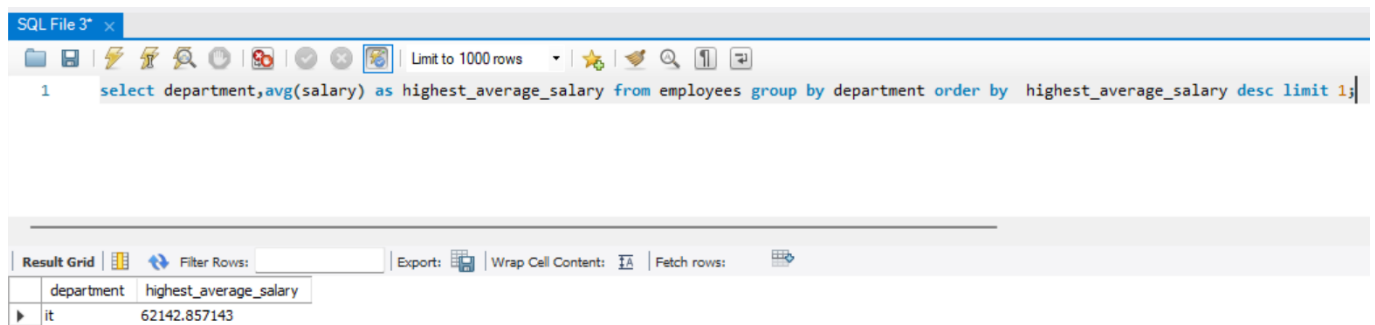
	emp_name	salary
▶	nani	20000.00

The IDE interface includes various icons for file operations, execution, and settings. The "Limit to 1000 rows" option is visible in the top right corner of the query editor.

Stage 11 – Challenge Questions

Challenge 5 – Department with Highest Average Salary

explanation: used group by, avg(), order by desc, and limit to find the department with the highest average salary. the result is it.



The screenshot shows a SQL IDE interface. At the top, there's a toolbar with various icons and a 'Limit to 1000 rows' dropdown. Below the toolbar, a SQL query is entered in a text area:

```
1 select department, avg(salary) as highest_average_salary from employees group by department order by highest_average_salary desc limit 1;
```

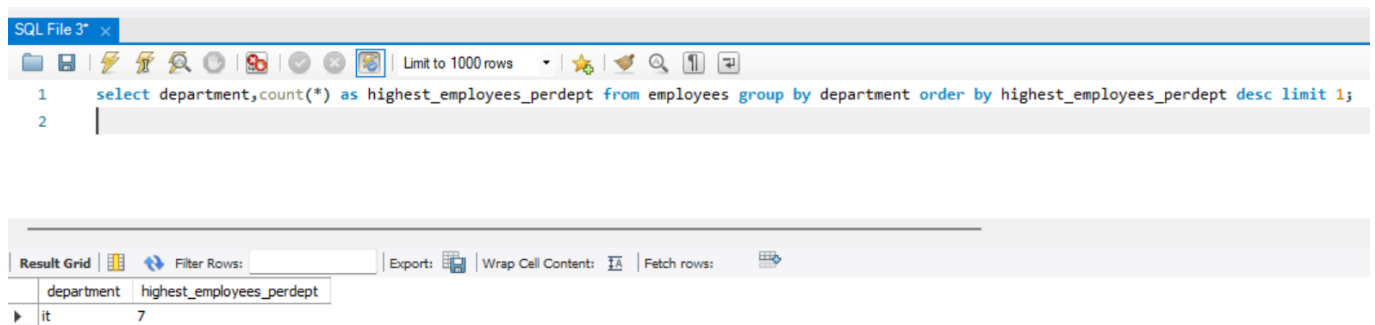
Below the query editor, there's a 'Result Grid' section. It includes a 'Filter Rows:' input field, an 'Export:' button, a 'Wrap Cell Content:' checkbox, and a 'Fetch rows:' input field. The result grid itself is a table with two columns: 'department' and 'highest_average_salary'. The first row shows the department 'it' with an average salary of 62142.857143.

department	highest_average_salary
it	62142.857143

Stage 11 – Challenge Questions

Challenge 6 – Department with Highest Number of Employees

explanation: used group by,count(),order by desc,and limit to find the department with the highest number of employees. it has 7 employees.



The screenshot shows a SQL IDE interface. The top panel displays a SQL query: `select department,count(*) as highest_employees_perdept from employees group by department order by highest_employees_perdept desc limit 1;`. The bottom panel shows the result grid with two columns: `department` and `highest_employees_perdept`. The first row shows the department `it` with a count of `7`.

```
1 select department,count(*) as highest_employees_perdept from employees group by department order by highest_employees_perdept desc limit 1;
2
```

department	highest_employees_perdept
it	7

Stage 11 – Challenge Questions

Challenge 7 – Employees Above Department Average

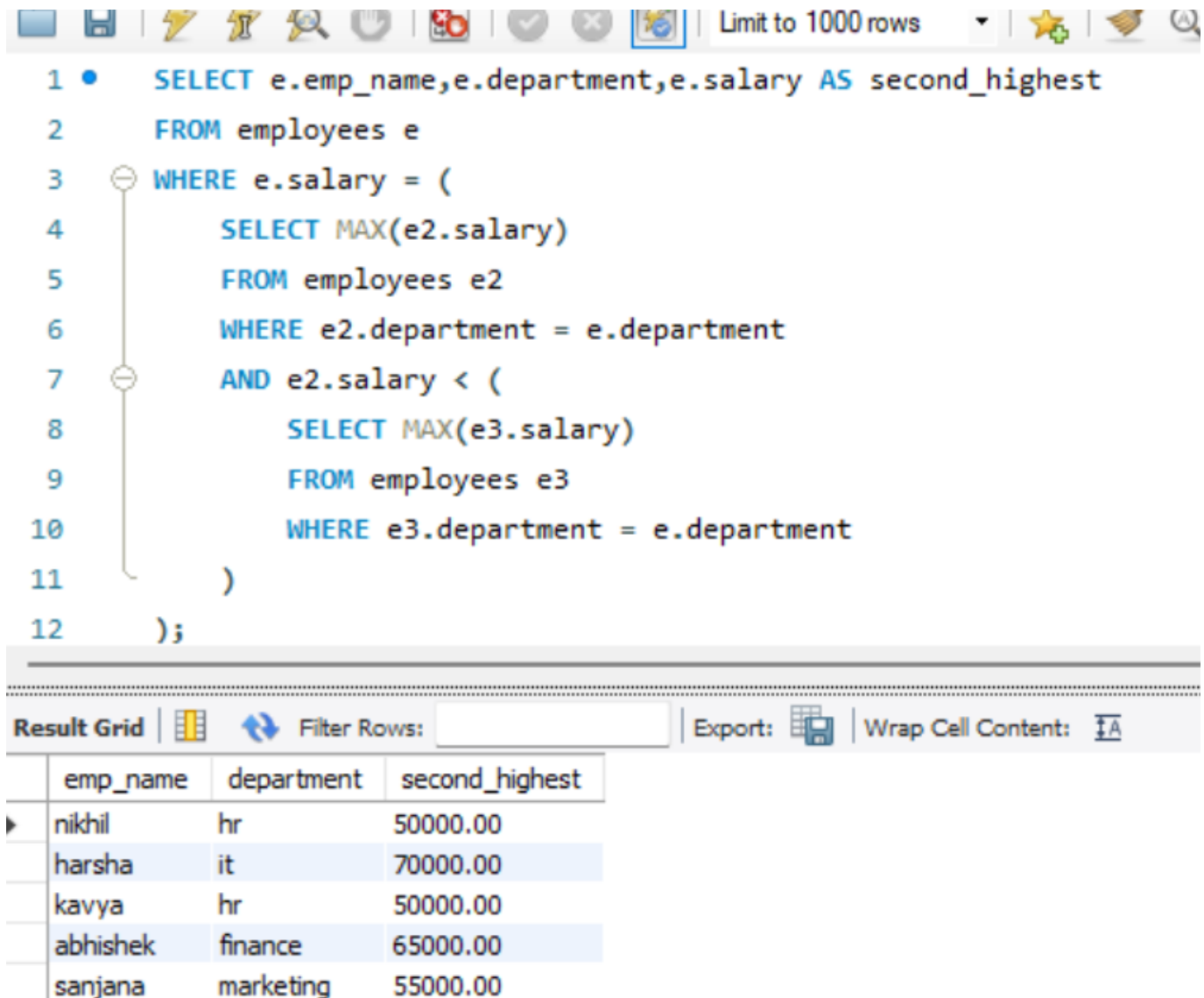
explanation: used a correlated subquery to display employees whose salary is greater than the average salary of their respective department.

```
SQL File 3* x
Limit to 1000 rows
1 • select emp_name,department,salary from employees as e where salary >(select avg(salary) from employees where department=e.department);
2
```

emp_name	department	salary
faizan	marketing	60000.00
nikhil	hr	50000.00
aishwarya	it	80000.00
harsha	it	70000.00
kavya	hr	50000.00
abhishek	finance	65000.00
ishita	hr	60000.00
rakesh	finance	75000.00
pranav	it	65000.00
madhuri	marketing	60000.00
varun	it	80000.00

Challenge 8 – Second Highest Salary in Each Department

explanation: used a correlated subquery to find the second-highest salary in each department and display the employee, department, and salary.



The screenshot shows a SQL IDE interface. At the top, there's a toolbar with various icons and a dropdown menu set to "Limit to 1000 rows". Below the toolbar, a SQL query is entered in a text area, with line numbers 1 through 12 on the left. The query is as follows:

```

1 • SELECT e.emp_name,e.department,e.salary AS second_highest
2 FROM employees e
3 WHERE e.salary = (
4     SELECT MAX(e2.salary)
5     FROM employees e2
6     WHERE e2.department = e.department
7     AND e2.salary < (
8         SELECT MAX(e3.salary)
9         FROM employees e3
10        WHERE e3.department = e.department
11    )
12 );
  
```

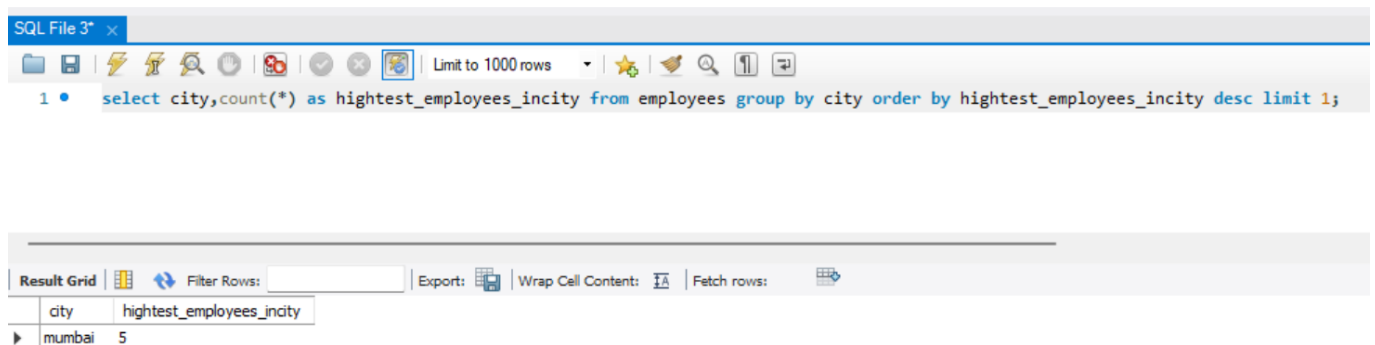
Below the query editor, there's a "Result Grid" section. It includes a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. The result grid itself is a table with the following data:

emp_name	department	second_highest
nikhil	hr	50000.00
harsha	it	70000.00
kavya	hr	50000.00
abhishek	finance	65000.00
sanjana	marketing	55000.00

Stage 11 – Challenge Questions

Challenge 9 – City with Highest Number of Employees

explanation: used group by,count(),order by desc,and limit to find the city with the highest number of employees. mumbai has 5 employees.



The screenshot shows a SQL IDE interface. At the top, there's a tab labeled "SQL File 3* x". Below the tab is a toolbar with various icons for file operations, execution, and editing. A dropdown menu shows "Limit to 1000 rows". The main area contains a single SQL query:

```
1 • select city,count(*) as highest_employees_incit from employees group by city order by highest_employees_incit desc limit 1;
```

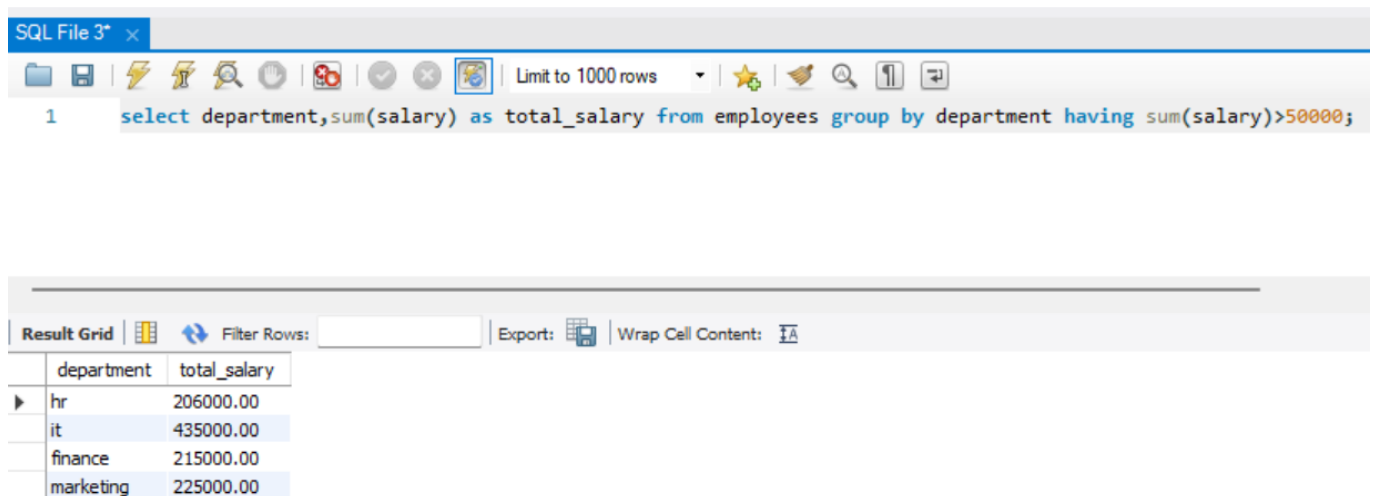
Below the query editor, there's a "Result Grid" section. It includes a "Filter Rows:" input field, an "Export:" button, a "Wrap Cell Content:" checkbox, and a "Fetch rows:" button. The result grid itself is a table with two columns: "city" and "highest_employees_incit". The first row shows "mumbai" and "5".

city	highest_employees_incit
mumbai	5

Stage 11 – Challenge Questions

Challenge 10 – Departments with Total Salary Above Rs.50,000

explanation: used group by with having and sum() to display departments whose total salary is greater than rs.50,000.



The screenshot shows a SQL IDE interface. At the top, there's a toolbar with various icons for file operations, execution, and navigation. Below the toolbar, a SQL query is entered in a text area:

```
1 select department,sum(salary) as total_salary from employees group by department having sum(salary)>50000;
```

Below the query editor, the 'Result Grid' is displayed, showing the output of the query. The grid has two columns: 'department' and 'total_salary'. The results are as follows:

department	total_salary
hr	206000.00
it	435000.00
finance	215000.00
marketing	225000.00